

DATA 260 Homework 4

React, MySQL, Query Tuning, and Grounded RAG

Vedant Vartak
Grocery Supply and Recall Notices

Personal configuration

Item	Value
SID4	3539
PORT_BASE	8839
PREFIX	s3539
SEED	3539
VERIFY_SEED	263539
DOMAIN_ID	3
Assigned domain	Grocery Supply and Recall Notices
Hardware	Apple M4 MacBook Air, 10 CPU cores, 16 GB memory
Local LLM	qwen3:8b
Embedding model	sentence-transformers/all-MiniLM-L6-v2
Code and results commit	8753d1f50e8e9384df50d30a1277c095ce85cbde
Final submission tag	hw4

GitHub repository: <https://github.com/vedantvartakwork/data260-3539>

The repository link was checked, and both required collaborators, Sbnikitha and supriyaselvangesan, have access. The Canvas copy is named Vartak_HW4.pdf.

This submission extends the existing course repository with a protected React client, FastAPI and MySQL persistence, measured N+1 query behavior, an index experiment, and a grounded local RAG evaluation. The final smoke test passed all 11 objective checks.

Part 1 React client

The client uses React Router, hooks, props, and protected routes. Login state is recovered from the backend session, and every create, update, and delete operation is persisted through the API before the user returns to the home page.

1.1 Login and protected routing

The login component stores the email and password with `useState` and passes them to the parent handler. `ProtectedRoute` redirects unauthenticated users to `/login`.

Login.jsx and App.jsx

```
export default function Login({ onLogin }) {
  const [email, setEmail] = useState("admin@example.edu");
  const [password, setPassword] = useState("password");
  const [error, setError] = useState("");
  const [submitting, setSubmitting] = useState(false);

  async function handleSubmit(event) {
    event.preventDefault();
    setSubmitting(true);
    setError("");
    try {
      await onLogin(email, password);
    } catch (loginError) {
      setError(loginError.message);
    } finally {
      setSubmitting(false);
    }
  }

  function ProtectedRoute({ user, checkingSession, children }) {
    if (checkingSession) {
      return <div className="state-card">Checking your session...</div>;
    }
    return user ? children : <Navigate to="/login" replace />;
  }
}
```

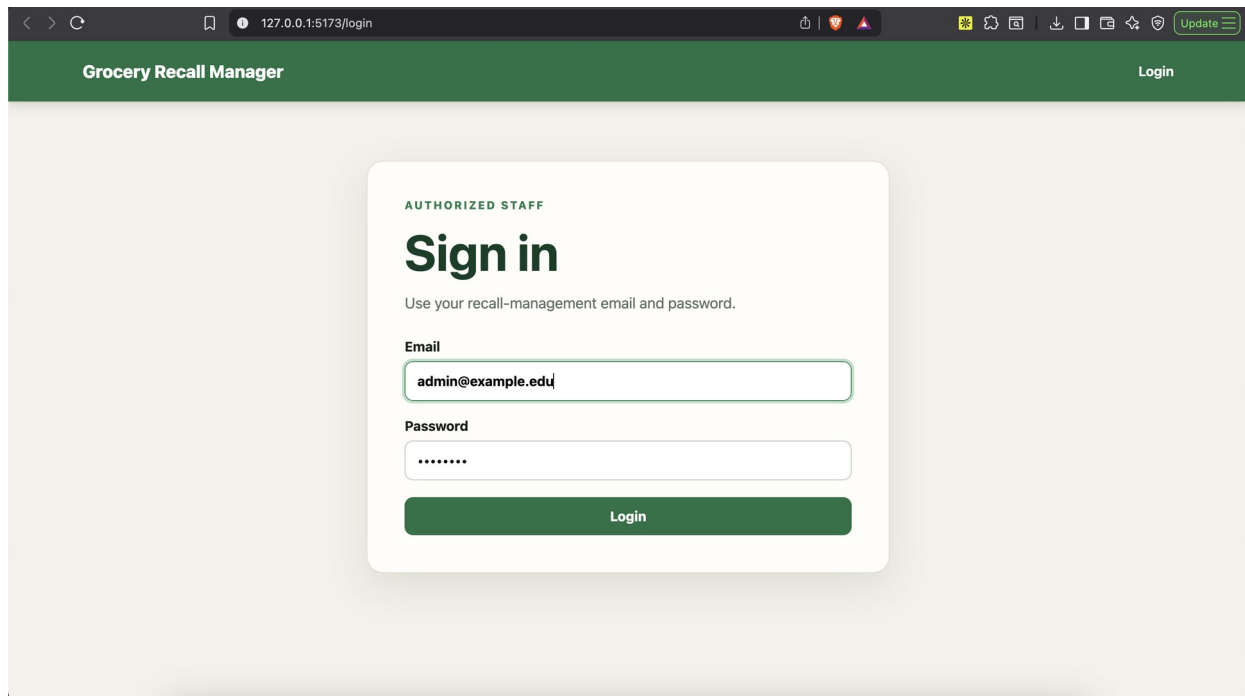


Figure 1. Login page with email and password fields.

1.2 Home record list

Home receives records and the authenticated user through props. It renders the current recall records and routes each row to update and delete actions.

Home.jsx

```
{!loading && !error && records.length > 0 && (
  <div className="record-grid">
    {records.map((record) => (
      <article className="record-card" key={record.id}>
        <div className="record-title-row">
          <div>
            <span className="record-id">#{record.id}</span>
            <h3>{record.product_name}</h3>
          </div>
          <span className="category-pill">{record.category}</span>
        </div>
        <p className="brand">{record.brand_name}</p>
        <p>{record.recall_details}</p>
        <p className="related-count">Related events: {record.events.length}</p>
        <div className="actions">
          <Link className="secondary button-link" to={` /update/${record.id}`}>Update</Link>
          <Link className="danger button-link" to={` /delete/${record.id}`}>Delete</Link>
        </div>
      </article>
    ))}
  </div>
```

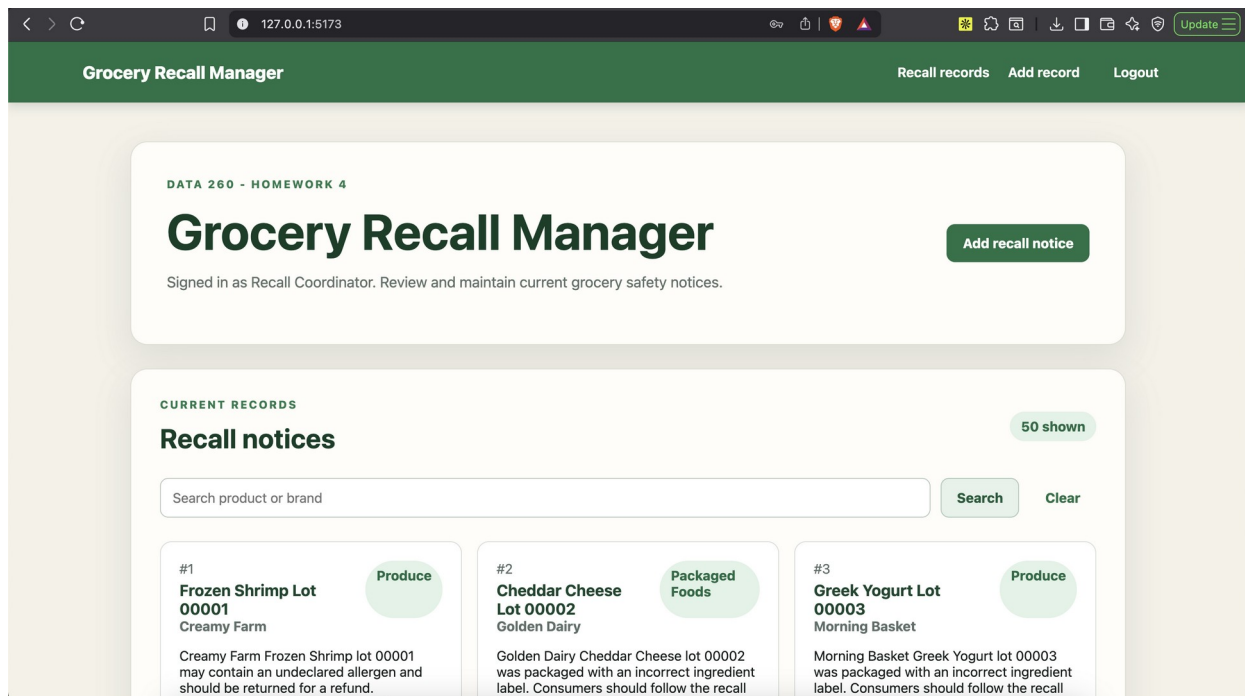


Figure 2. Protected home page showing recall records and related-event counts.

1.3 Create record form

CreateRecord receives the create callback as a prop and reuses the validated RecordForm component.

CreateRecord.jsx

```
export default function CreateRecord({ onCreate }) {  
  return (  
    <section className="panel narrow">  
      <p className="eyebrow">Create</p>  
      <h1>Add a recall notice</h1>  
      <p className="intro">Enter the affected product and recall information.</p>  
      <RecordForm submitLabel="Add recall notice" onSubmit={onCreate} />  
    </section>  
  );  
}
```

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5173/create'. The application has a dark green header with the title 'Grocery Recall Manager' and navigation links for 'Recall records', 'Add record', and 'Logout'. The main content area features a light green card titled 'CREATE' with the heading 'Add a recall notice'. Below the heading is a subheading 'Enter the affected product and recall information.' The form contains several input fields: 'Product name' and 'Brand or manufacturer' (text inputs), 'Submitter email' (text input), and 'Product category' (a dropdown menu with 'Select a category' as the placeholder). There is also a large text area for 'Recall details'. At the bottom of the form, there is a checkbox labeled 'I agree to the terms and conditions.' and a green button labeled 'Add recall notice'.

Figure 3. Create-record route and domain-specific form.

1.4 Create operation result

After the POST succeeds, the client reloads the list, sets a success message with the new auto-incremented ID, and redirects to the root route.

App.jsx

```
async function handleCreate(payload) {
  const created = await recallsApi.create(payload);
  await loadRecords();
  setNotice(`Added recall ID ${created.id} successfully.`);
  navigate("/");
}
```

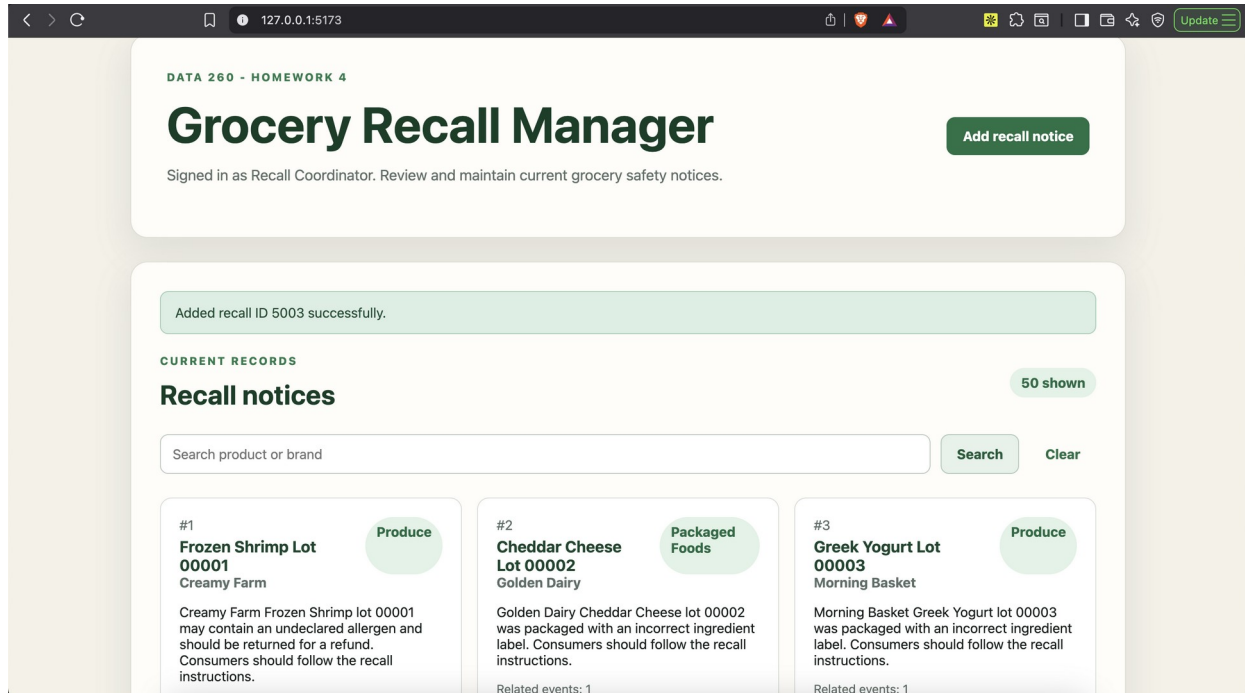


Figure 4. Successful creation of recall ID 5003.

1.5 Update record form

The update component reads the route ID, loads the current record in `useEffect`, and passes the edited payload back through the `onUpdate` prop.

UpdateRecord.jsx

```
export default function UpdateRecord({ onUpdate }) {
  const { id } = useParams();
  const [record, setRecord] = useState(null);
  const [error, setError] = useState("");

  useEffect(() => {
    recallsApi.get(id).then(setRecord).catch((loadError) => setError(loadError.message));
  }, [id]);

  if (error) return <div className="alert error">{error}</div>;
  if (!record) return <div className="state-card">Loading recall notice...</div>;

  return (
    <section className="panel narrow">
      <p className="eyebrow">Update</p>
      <h1>Update recall #{id}</h1>
      <p className="intro">Edit the record and save it back to MySQL.</p>
      <RecordForm
        initialRecord={record}
        submitLabel="Update recall notice"
        onSubmit={(payload) => onUpdate(Number(id), payload)}
      />
    </section>
  );
}
```

The screenshot shows a web browser window with the URL `127.0.0.1:5173/update/5003`. The application is titled "Grocery Recall Manager" and has navigation links for "Recall records", "Add record", and "Logout". The main content area displays the "UPDATE" form for recall #5003. The form includes the following fields:

- Product name:** Sunrise Oat Bites
- Brand or manufacturer:** Campus Pantry Updated
- Submitter email:** student@example.edu
- Product category:** Packaged Foods (selected from a dropdown menu)
- Recall details:** A text area containing the text: "Selected packages may contain an undeclared peanut allergen. Customers should return them for a full refund."
- Terms and conditions:** A checkbox labeled "I agree to the terms and conditions." which is checked.

At the bottom of the form is a green button labeled "Update recall notice".

Figure 5. Update form populated from MySQL for recall ID 5003.

1.6 Update operation result

The update handler writes the record through the backend, refreshes the list, and redirects to the home route with confirmation.

App.jsx

```
async function handleUpdate(id, payload) {  
  await recallsApi.update(id, payload);  
  await loadRecords();  
  setNotice(`Updated recall ID ${id} successfully.`);  
  navigate("/");  
}
```

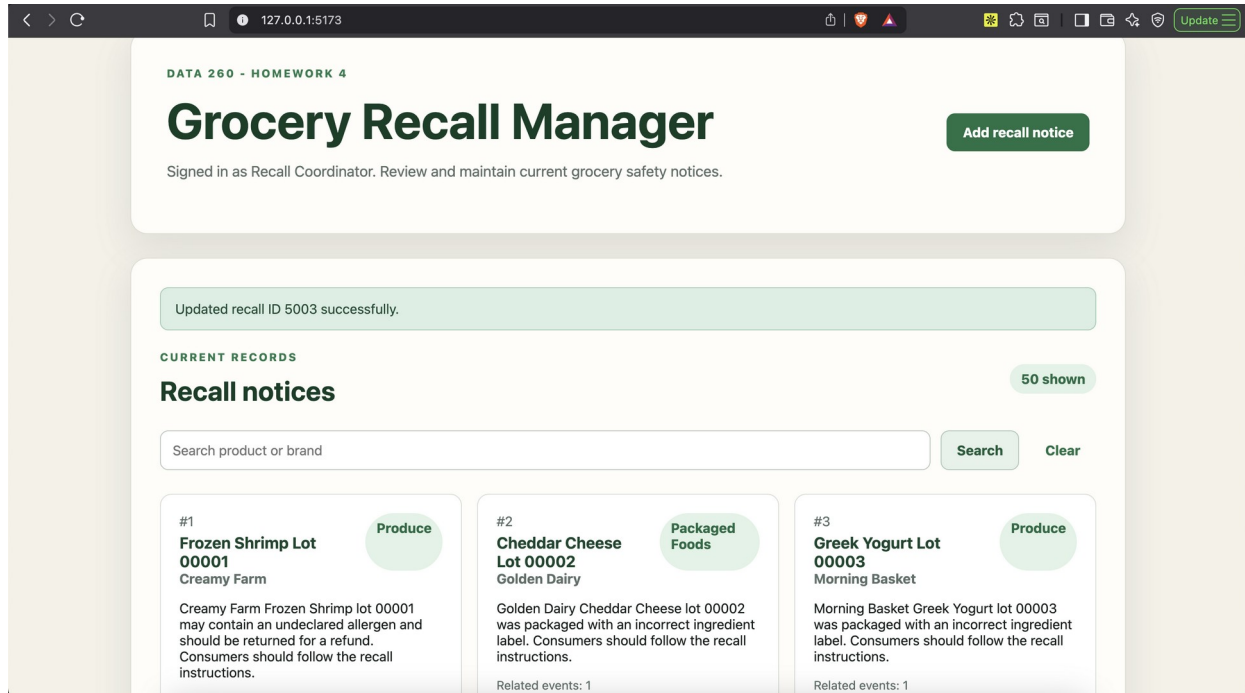


Figure 6. Successful update confirmation on the home page.

1.7 Delete confirmation

The delete route loads the selected record and requires a deliberate button click before calling the parent delete handler.

DeleteRecord.jsx

```
export default function DeleteRecord({ onDelete }) {
  const { id } = useParams();
  const [record, setRecord] = useState(null);
  const [error, setError] = useState("");

  useEffect(() => {
    recallsApi.get(id).then(setRecord).catch((loadError) => setError(loadError.message));
  }, [id]);

  if (error) return <div className="alert error">{error}</div>;
  if (!record) return <div className="state-card">Loading recall notice...</div>;

  return (
    <section className="panel narrow">
      <p className="eyebrow">Delete</p>
      <h1>Delete recall #{id}</h1>
      <p className="intro">
        Remove <strong>{record.product_name}</strong> by {record.brand_name} from MySQL?
      </p>
      <div className="actions">
        <button className="danger" onClick={() => onDelete(Number(id))}>Delete recall notice</button>
        <Link className="secondary button-link" to="/">Cancel</Link>
      </div>
    </section>
  );
}
```

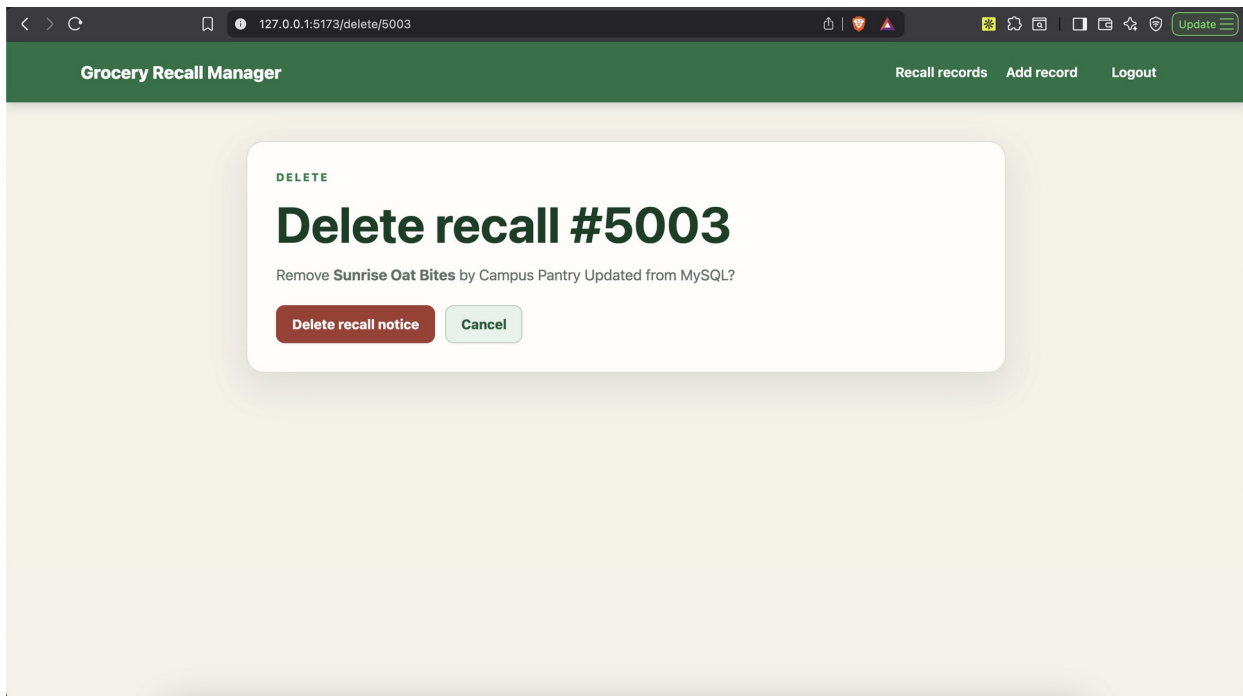


Figure 7. Delete confirmation for recall ID 5003.

1.8 Delete operation result

After the DELETE request succeeds, the client reloads the list and shows the deleted ID in a success message.

App.jsx

```
async function handleDelete(id) {  
  await recallsApi.remove(id);  
  await loadRecords();  
  setNotice(`Deleted recall ID ${id} successfully.`);  
  navigate("/");  
}
```

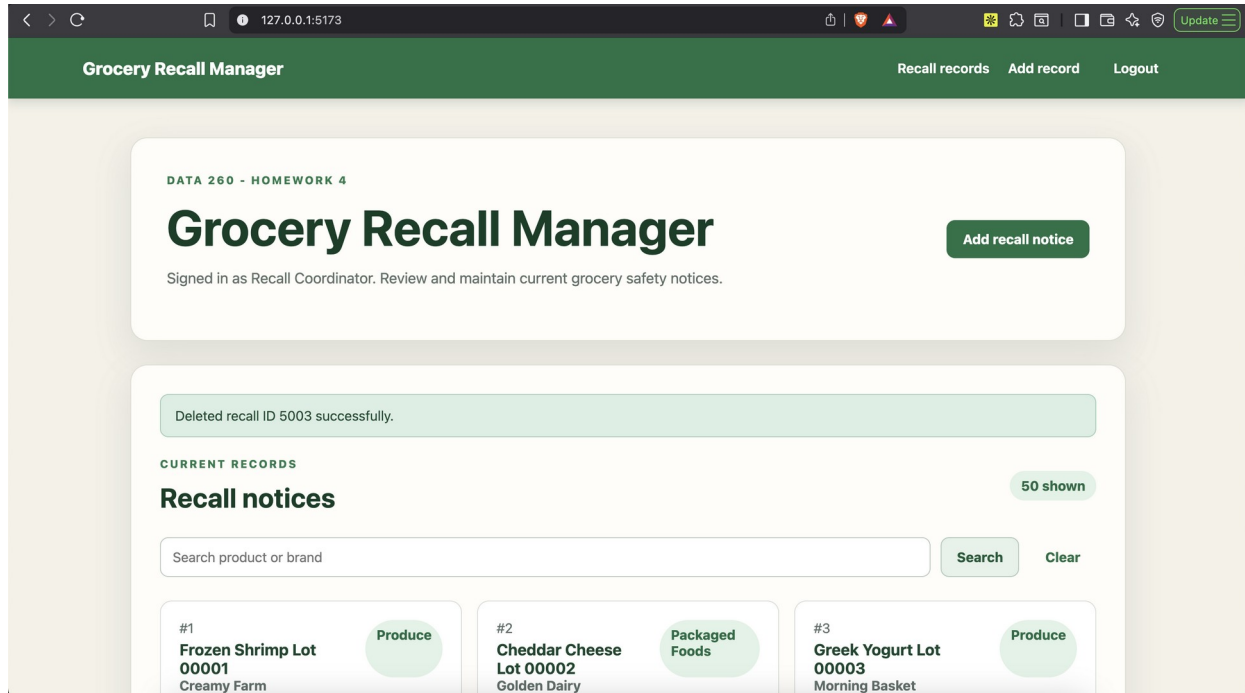


Figure 8. Successful deletion confirmation on the home page.

Part 2 MySQL persistence and server-side sessions

The FastAPI service uses SQLAlchemy with the required `db_session_base` variable and the `s3539_rel` MySQL database. Passwords are stored as hashes. Login creates a random opaque token in the sessions table and sends only that token in an HTTP-only SameSite cookie. Every recall endpoint depends on `require_user`.

2.1 Database connection and required variable

The connection targets `s3539_rel` and exposes the exact required SQLAlchemy session variable name.

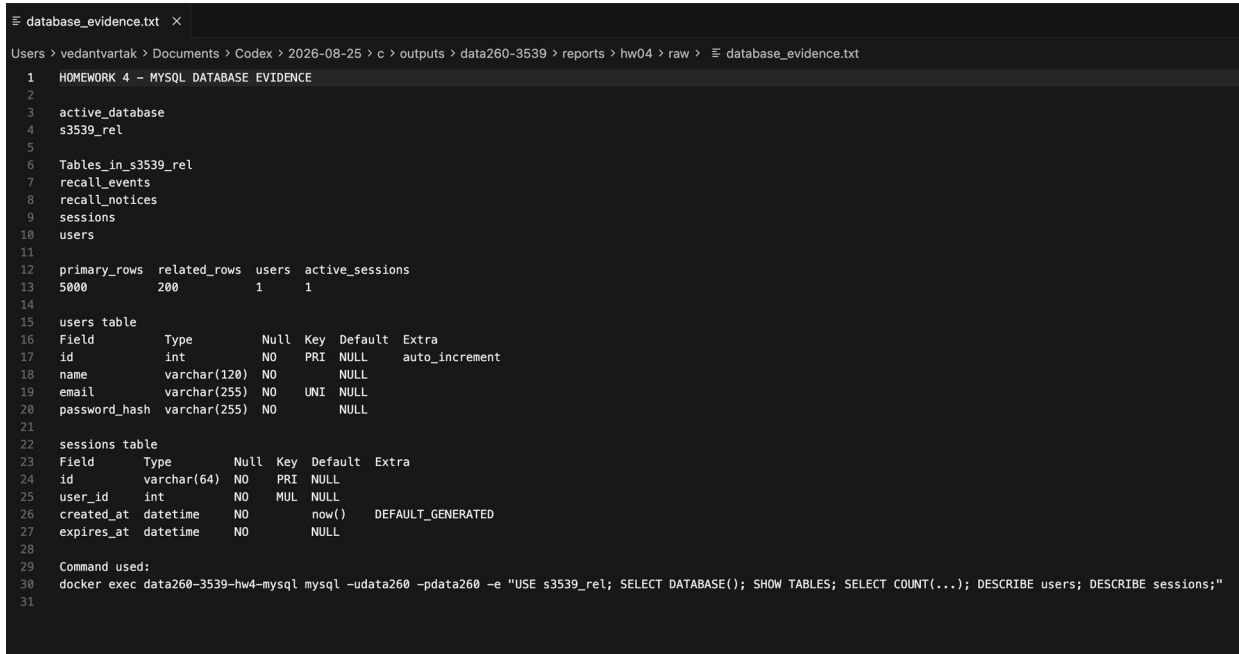
```
db.py
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "mysql+pymysql://data260:data260@127.0.0.1:3307/s3539_rel",
)

engine = create_engine(DATABASE_URL, pool_pre_ping=True)

# The assignment requires this exact connection-variable name.
db_session_base = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    pass

def get_db() -> Generator[Session, None, None]:
    db = db_session_base()
    try:
        yield db
    finally:
        db.close()
```



```
database_evidence.txt
Users > vedantvartak > Documents > Codex > 2026-08-25 > c > outputs > data260-3539 > reports > hw04 > raw > database_evidence.txt

1  HOMEWORK 4 - MYSQL DATABASE EVIDENCE
2
3  active_database
4  s3539_rel
5
6  Tables_in_s3539_rel
7  recall_events
8  recall_notices
9  sessions
10 users
11
12 primary_rows  related_rows  users  active_sessions
13 5000          200          1      1
14
15 users table
16 Field      Type      Null Key  Default  Extra
17 id         int       NO   PRI  NULL    auto_increment
18 name       varchar(120) NO   NULL
19 email      varchar(255) NO   UNI  NULL
20 password_hash varchar(255) NO   NULL
21
22 sessions table
23 Field      Type      Null Key  Default  Extra
24 id         varchar(64) NO   PRI  NULL
25 user_id    int       NO   MUL  NULL
26 created_at datetime   NO   now()   DEFAULT_GENERATED
27 expires_at datetime   NO   NULL
28
29 Command used:
30 docker exec data260-3539-hw4-mysql mysql -udata260 -pdata260 -e "USE s3539_rel; SELECT DATABASE(); SHOW TABLES; SELECT COUNT(...); DESCRIBE users; DESCRIBE sessions;"
31
```

Figure 9. MySQL database, table, row-count, user, and session evidence.

2.2 Project folder structure

All application code remains in shared root-level folders and extends the existing repository rather than creating a separate homework copy.

Command used to generate the evidence

```
find code src sql scripts tests reports/hw04 -maxdepth 4 -type f | sort
```

```
1 data260-3539/
2 |
3 |   code/
4 |   |   web_application/
5 |   |   |   backend.py
6 |   |   |   db.py
7 |   |   |   models.py
8 |   |   |   query_counter.py
9 |   |   |   schemas.py
10 |   |   |   security.py
11 |   |   |   routers/
12 |   |   |   |   api_auth.py
13 |   |   |   |   recalls.py
14 |   |   |   frontend/
15 |   |   |   |   package.json
16 |   |   |   |   |   src/
17 |   |   |   |   |   |   App.jsx
18 |   |   |   |   |   |   api.js
19 |   |   |   |   |   |   components/
20 |   |   |   |   |   |   pages/
21 |   |   |   src/
22 |   |   |   |   model_client.py
23 |   |   |   sql/
24 |   |   |   |   hw04_schema.sql
25 |   |   |   scripts/
26 |   |   |   |   |   seed_hw04.py
27 |   |   |   |   |   run_hw4_plus1_experiment.py
28 |   |   |   |   |   run_hw4_explain.py
29 |   |   |   |   |   generate_hw4_metrics.py
30 |   |   |   |   |   verify_hw04.py
31 |   |   |   tests/
32 |   |   |   |   test_hw4_structure.py
33 |   |   |   reports/
34 |   |   |   |   hw04/
35 |   |   |   |   |   raw/
36 |   |   |   |   |   |   postman/
37 |   |   |   |   |   |   |   METRICS.md
38 |   |   |   |   |   |   |   RAG_ANALYSIS.md
39 |   |   |   |   |   |   |   questions.yaml
40 |   |   |   |   |   |   |   verification.json
41 |   |   |   docker-compose.hw4.yml
42 |   |   |   rag.py
43 |   |   |   Makefile
44 |   |   |   requirements.txt
```

Figure 10. Project structure covering backend, React, SQL, scripts, tests, and report artifacts.

2.3 Login API and opaque session cookie

Login verifies the password hash, replaces any earlier session, stores a random token server-side, and marks the cookie HTTP-only.

```
api_auth.py
@router.post("/login", response_model=UserResponse)
def login(payload: LoginRequest, response: Response, db: Session = Depends(get_db)) -> User:
    user = db.scalar(
        select(User).where(User.email == payload.email.lower())
    )
    if user is None or not verify_password(payload.password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid email or password",
        )

    db.execute(delete(SessionRecord).where(SessionRecord.user_id == user.id))
    session = SessionRecord(
        id=secrets.token_hex(32),
        user_id=user.id,
        expires_at=utc_now() + timedelta(minutes=SESSION_TTL_MINUTES),
    )
    db.add(session)
    db.commit()

    secure_cookie = os.getenv("SESSION_COOKIE_SECURE", "false").lower() in {
        "1",
        "true",
        "yes",
    }
    response.set_cookie(
        key=COOKIE_NAME,
        value=session.id,
        httponly=True,
        secure=secure_cookie,
        samesite="lax",
        max_age=SESSION_TTL_MINUTES * 60,
        path="/",
    )
    return user
```

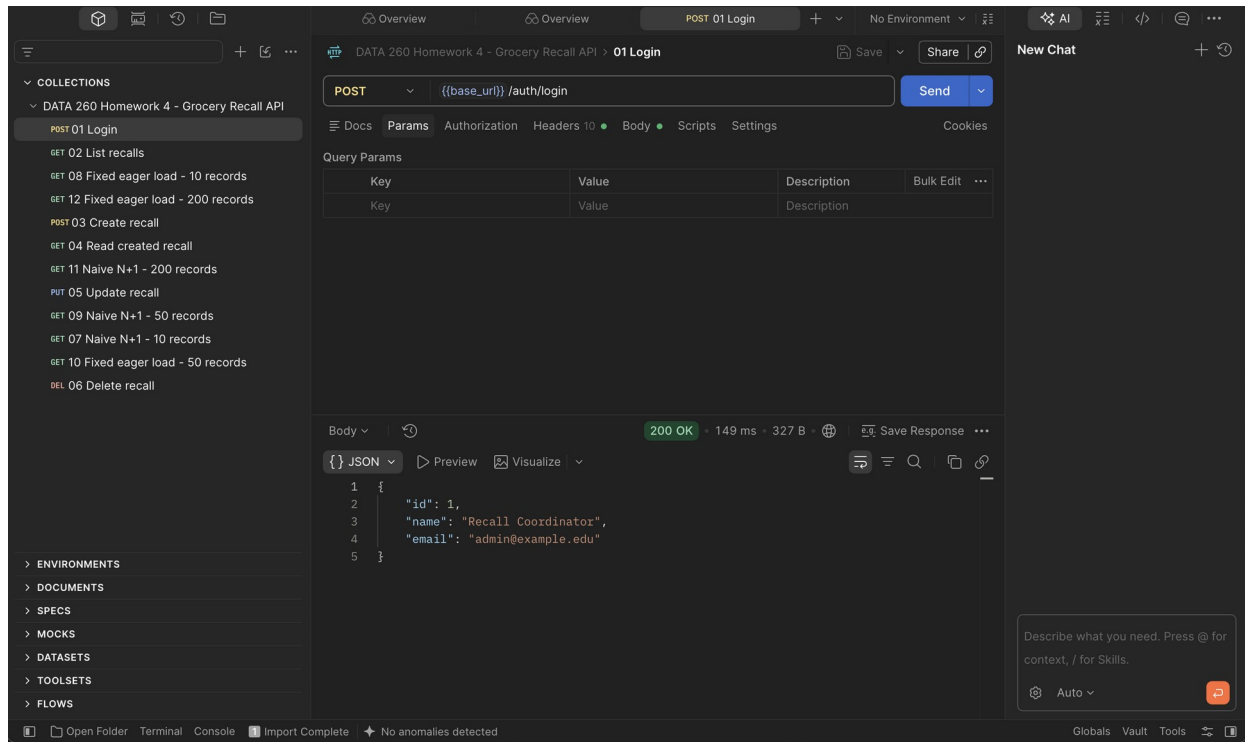


Figure 11. Postman login response with authenticated user data.

2.4 View all records

The protected list endpoint eager-loads related events and supports a bounded page size and optional product or brand search.

```
recalls.py
@router.get("", response_model=RecallListResponse)
def list_recalls(
    response: Response,
    q: str = Query(default="", max_length=160),
    page_size: int = Query(default=50, ge=1, le=200),
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> RecallListResponse:
    statement = (
        select(RecallNotice)
        .options(joinedload(RecallNotice.events))
        .order_by(RecallNotice.id)
        .limit(page_size)
    )
    needle = q.strip()
    if needle:
        pattern = f"%{needle}%"
        statement = statement.where(
            or_(
                RecallNotice.product_name.like(pattern),
                RecallNotice.brand_name.like(pattern),
            )
        )

    with count_sql_queries() as counter:
        recalls = list(db.execute(statement).unique().scalars())
        payload = [serialize_recall(recall) for recall in recalls]
    response.headers["X-SQL-Query-Count"] = str(counter[0])
    return RecallListResponse(
        implementation="fixed",
        page_size=page_size,
        sql_queries=counter[0],
        records=payload,
    )
```

The screenshot shows a REST client interface with the following details:

- Endpoint:** GET `{{base_url}}/recalls?page_size=10`
- Method:** GET
- Status:** 200 OK, 88 ms, 5.26 KB
- Response Format:** JSON
- Response Body:**

```
{
  "implementation": "fixed",
  "page_size": 10,
  "sql_queries": 1,
  "records": [
    {
      "product_name": "Frozen Shrimp Lot 00001",
      "brand_name": "Creamy Farm",
      "submitter_email": "recall1@example.edu",
      "category": "Produce",
      "recall_details": "Creamy Farm Frozen Shrimp lot 00001 may contain an undeclared allergen and should be returned for a refund. Consumers should follow the recall instructions.",
      "terms_accepted": true
    }
  ]
}
```

Figure 12. GET recall list response with fixed implementation and one SQL statement.

2.5 Create a record

The POST endpoint validates the payload, adds a RecallNotice, commits it, refreshes its auto-incremented ID, and returns HTTP 201.

```
recalls.py
@router.post("", response_model=RecallResponse, status_code=201)
def create_recall(
    payload: RecallCreate,
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> RecallResponse:
    row = RecallNotice(**payload.model_dump())
    db.add(row)
    db.commit()
    db.refresh(row)
    return serialize_recall(row, [])
```

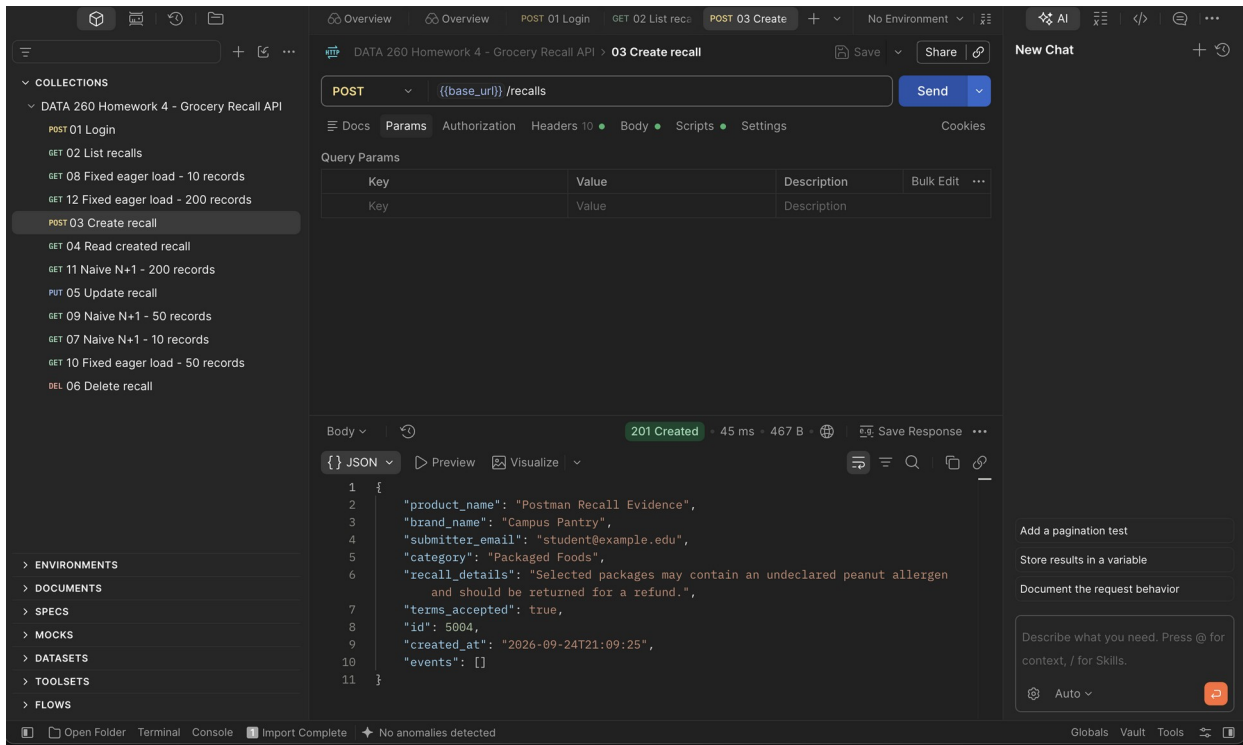


Figure 13. POST response for newly created recall ID 5004.

2.6 View a record by ID

The ID endpoint eager-loads events and returns 404 if the record does not exist.

```
recalls.py
@router.get("/{recall_id}", response_model=RecallResponse)
def get_recall(
    recall_id: int,
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> RecallResponse:
    row = db.execute(
        select(RecallNotice)
        .options(joinedload(RecallNotice.events))
        .where(RecallNotice.id == recall_id)
    ).unique().scalar_one_or_none()
    if row is None:
        raise HTTPException(status_code=404, detail="Recall notice not found")
    return serialize_recall(row)
```

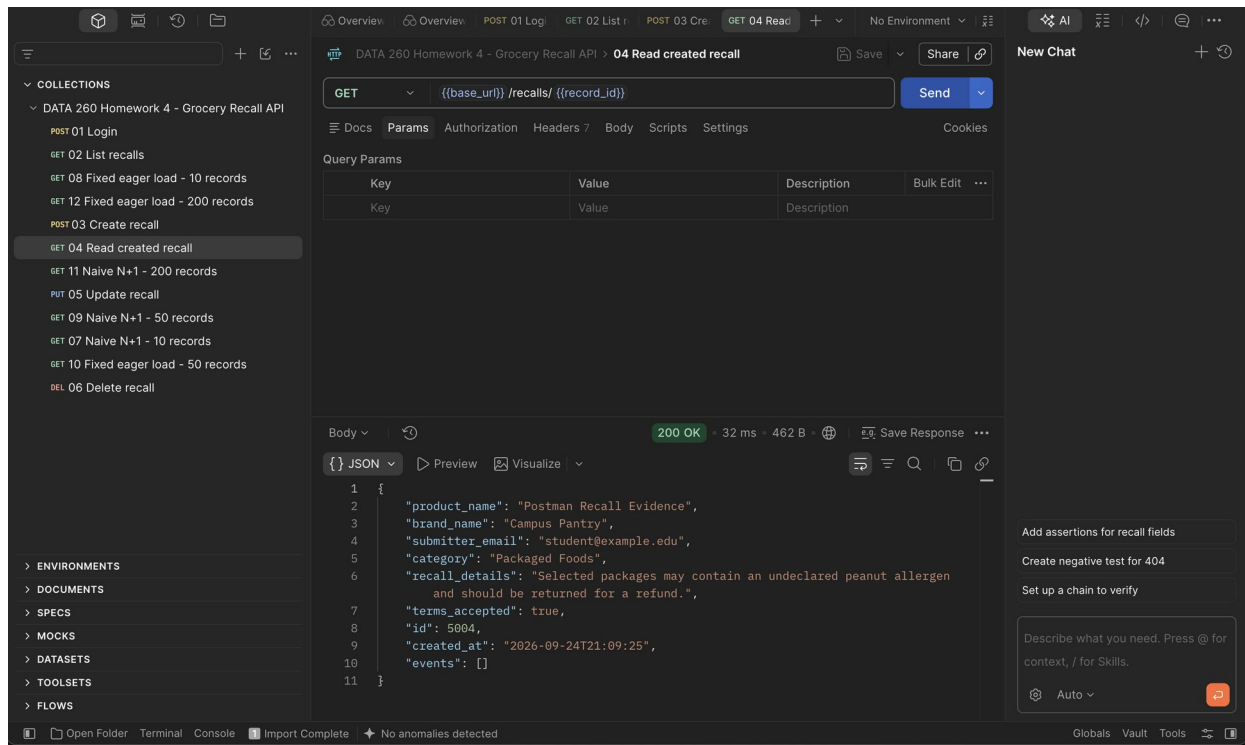


Figure 14. GET response for the created recall.

2.7 Update a record

The PUT endpoint loads the row, applies validated fields, commits, and returns the current related-event list.

```
recalls.py
@router.put("/{recall_id}", response_model=RecallResponse)
def update_recall(
    recall_id: int,
    payload: RecallUpdate,
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> RecallResponse:
    row = db.get(RecallNotice, recall_id)
    if row is None:
        raise HTTPException(status_code=404, detail="Recall notice not found")
    for field, value in payload.model_dump().items():
        setattr(row, field, value)
    db.commit()
    db.refresh(row)
    events = list(
        db.scalars(
            select(RecallEvent)
            .where(RecallEvent.recall_id == recall_id)
            .order_by(RecallEvent.id)
        )
    )
    return serialize_recall(row, events)
```

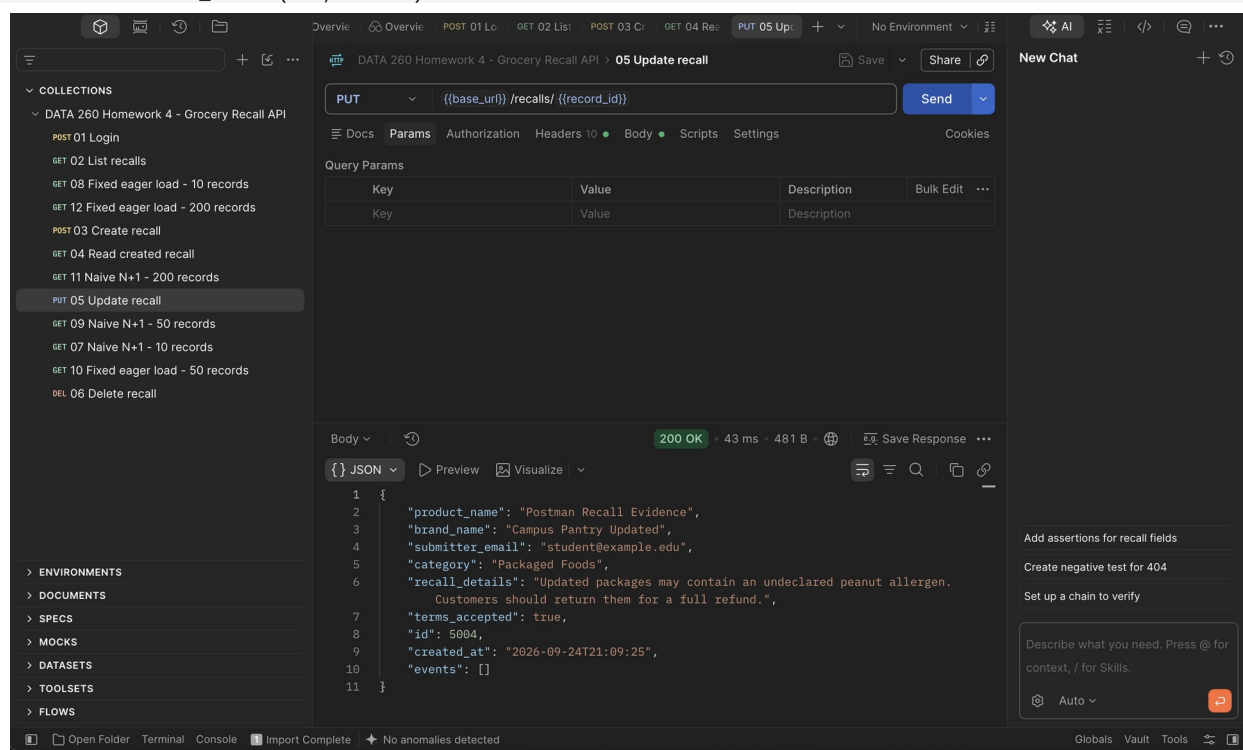


Figure 15. PUT response showing the updated brand and recall details.

2.8 Delete a record

The DELETE endpoint removes the requested row, returns 404 when no row was affected, and otherwise returns HTTP 204.

```
recalls.py
@router.delete("/{recall_id}", status_code=204)
def delete_recall(
    recall_id: int,
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> None:
    deleted = db.execute(
        delete(RecallNotice).where(RecallNotice.id == recall_id)
    )
    if deleted.rowcount == 0:
        db.rollback()
        raise HTTPException(status_code=404, detail="Recall notice not found")
    db.commit()
```

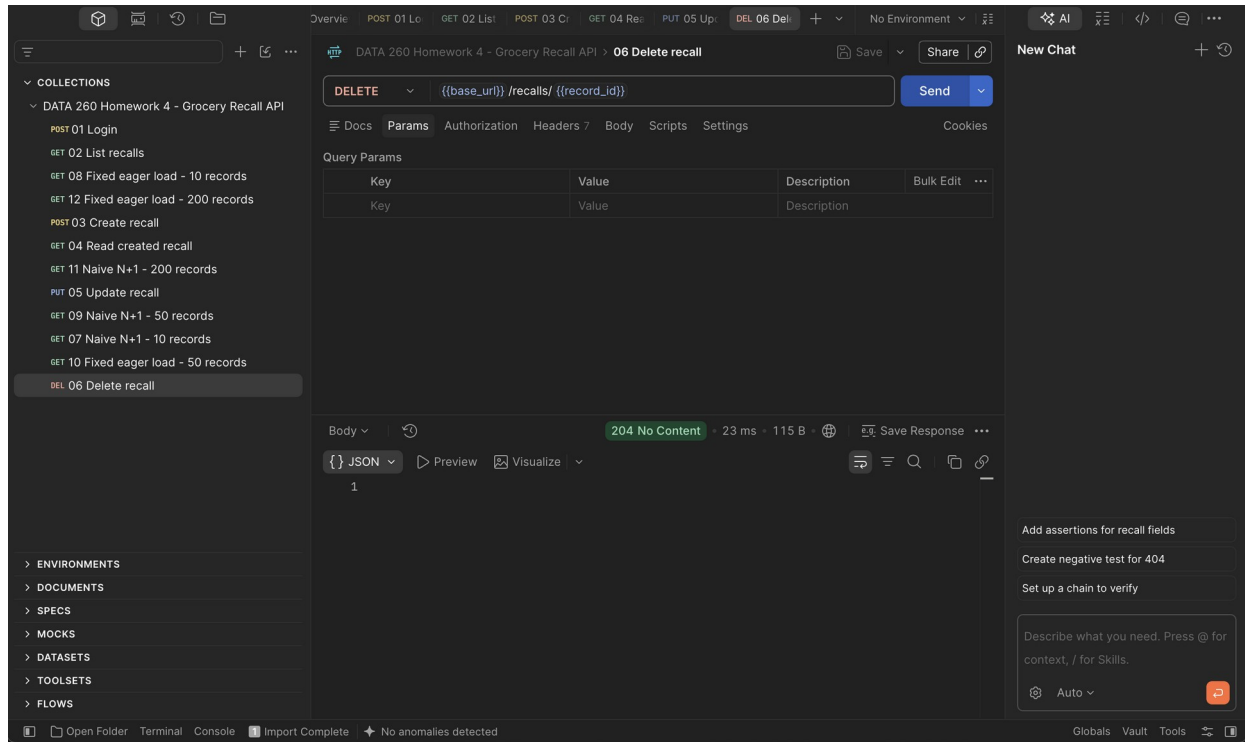


Figure 16. DELETE response with HTTP 204 No Content.

Part 3 N plus 1 measurement and query tuning

The deterministic seed script creates 5,000 recall notices and 200 related recall events with SEED 3539. The naive endpoint runs one main query plus one event query per returned row. The fixed endpoint eager-loads the relationship in one SQL statement. Each implementation was measured 30 times at page sizes 10, 50, and 200, giving 180 saved raw requests.

seed_hw04.py

```
SEED = 3539
PRIMARY_ROWS = 5_000
RELATED_ROWS = 200
PRODUCTS = [
    "Baby Spinach",
    "Greek Yogurt",
    "Peanut Butter",
    "Frozen Shrimp",
    ...
]
def main() -> None:
    rng = random.Random(SEED)
    Base.metadata.create_all(engine)

    with db_session_base26() as db:
        db.execute(delete(SessionRecord))
        db.execute(delete(RecallEvent))
        db.execute(delete(RecallNotice))
        db.execute(delete(User))
        db.commit()

        admin = User(
            name="Recall Coordinator",
            email="admin@example.edu",
            password_hash=hash_password("password"),
        )
        db.add(admin)
        db.add_all(make_recalls(rng))
        db.commit()
```

The database screenshot in Figure 9 verifies the seeded counts and authentication tables.

3.1 Naive endpoint at page size 10

For each returned recall, the loop sends a separate SELECT for recall_events. Ten rows therefore require 11 SQL statements.

```
recalls.py
@router.get("/naive", response_model=RecallListResponse)
def list_recalls_naive(
    response: Response,
    page_size: int = Query(default=10, ge=1, le=200),
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> RecallListResponse:
    with count_sql_queries() as counter:
        recalls = list(
            db.scalars(
                select(RecallNotice)
                .order_by(RecallNotice.id)
                .limit(page_size)
            )
        )
        payload = []
        for recall in recalls:
            events = list(
                db.scalars(
                    select(RecallEvent)
                    .where(RecallEvent.recall_id == recall.id)
                    .order_by(RecallEvent.id)
                )
            )
            payload.append(serialize_recall(recall, events))

    response.headers["X-SQL-Query-Count"] = str(counter[0])
    return RecallListResponse(
        implementation="naive",
        page_size=page_size,
```

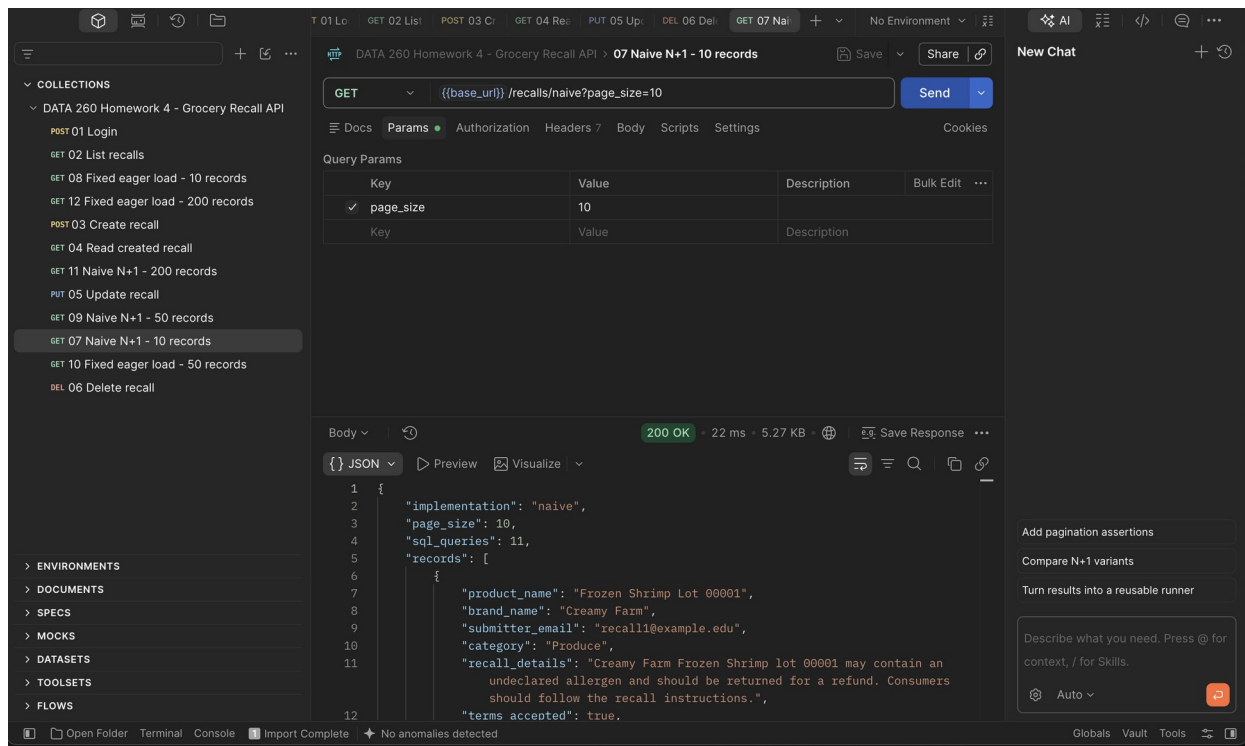


Figure 17. Naive endpoint at page size 10 with 11 SQL queries.

3.2 Fixed endpoint at page size 10

joinedload fetches the same relationship eagerly, so the query count remains one.

```
recalls.py
@router.get("/fixed", response_model=RecallListResponse)
def list_recalls_fixed(
    response: Response,
    page_size: int = Query(default=10, ge=1, le=200),
    db: Session = Depends(get_db),
    _user: User = Depends(require_user),
) -> RecallListResponse:
    with count_sql_queries() as counter:
        result = db.execute(
            select(RecallNotice)
            .options(joinedload(RecallNotice.events))
            .order_by(RecallNotice.id)
            .limit(page_size)
        )
        recalls = list(result.unique().scalars())
        payload = [serialize_recall(recall) for recall in recalls]

    response.headers["X-SQL-Query-Count"] = str(counter[0])
    return RecallListResponse(
        implementation="fixed",
        page_size=page_size,
        sql_queries=counter[0],
        records=payload,
    )
```

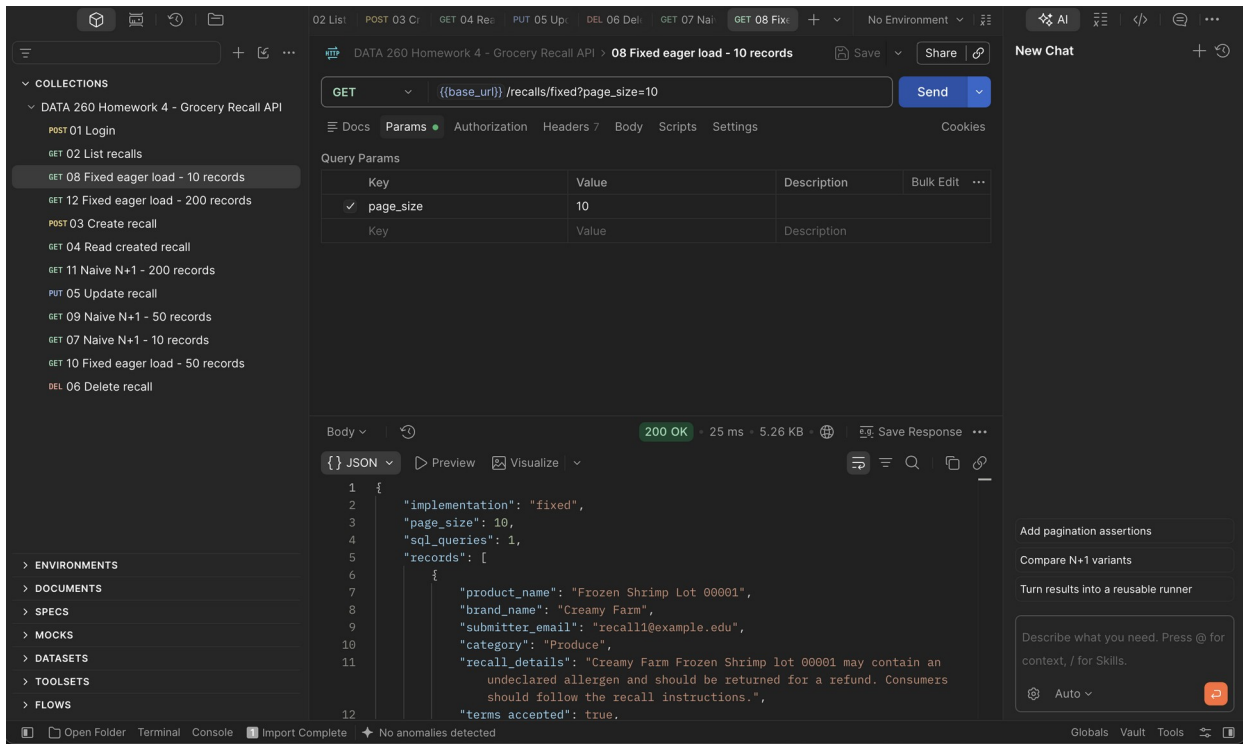


Figure 18. Fixed eager-loading endpoint at page size 10 with one SQL query.

3.3 Naive endpoint at page size 50

The N+1 pattern grows with the page: one recall query plus 50 related-event queries.

Measured request

```
GET /api/v1/recalls/naive?page_size=50
```

```
Expected SQL statements: 1 + page_size = 51
```

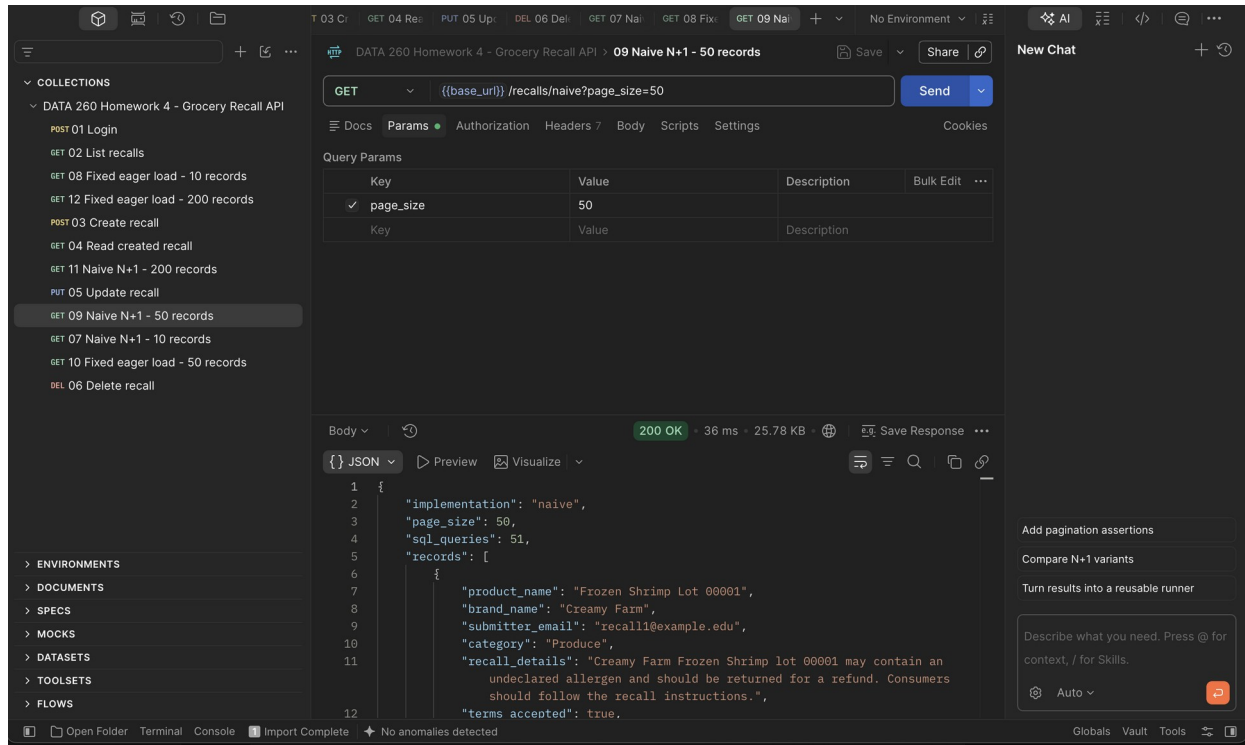


Figure 19. Naive endpoint at page size 50 with 51 SQL queries.

3.4 Fixed endpoint at page size 50

The eager-loading implementation returns the same record and event structure using one SQL statement.

Measured request

GET /api/v1/recalls/fixed?page_size=50
Expected SQL statements: 1

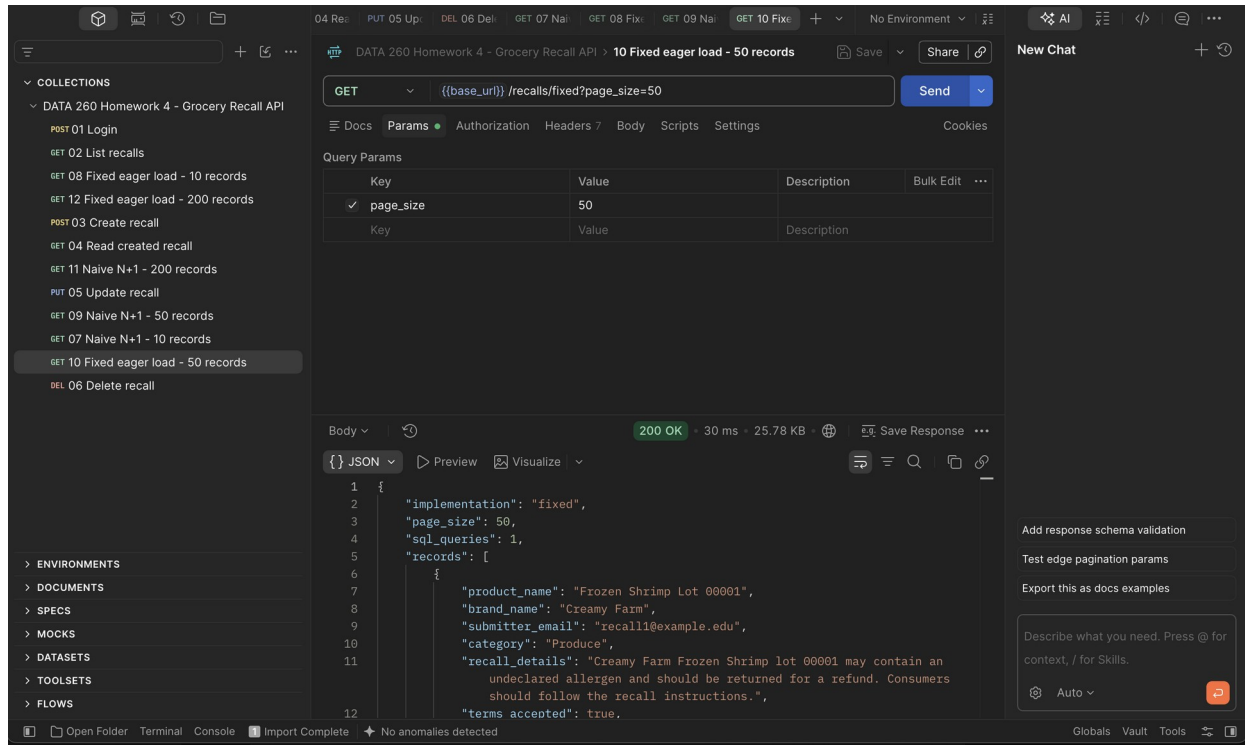


Figure 20. Fixed endpoint at page size 50 with one SQL query.

3.5 Naive endpoint at page size 200

At the largest page size, the naive endpoint issues 201 statements and shows the linear query-growth problem clearly.

Measured request

GET /api/v1/recalls/naive?page_size=200

Expected SQL statements: $1 + \text{page_size} = 201$

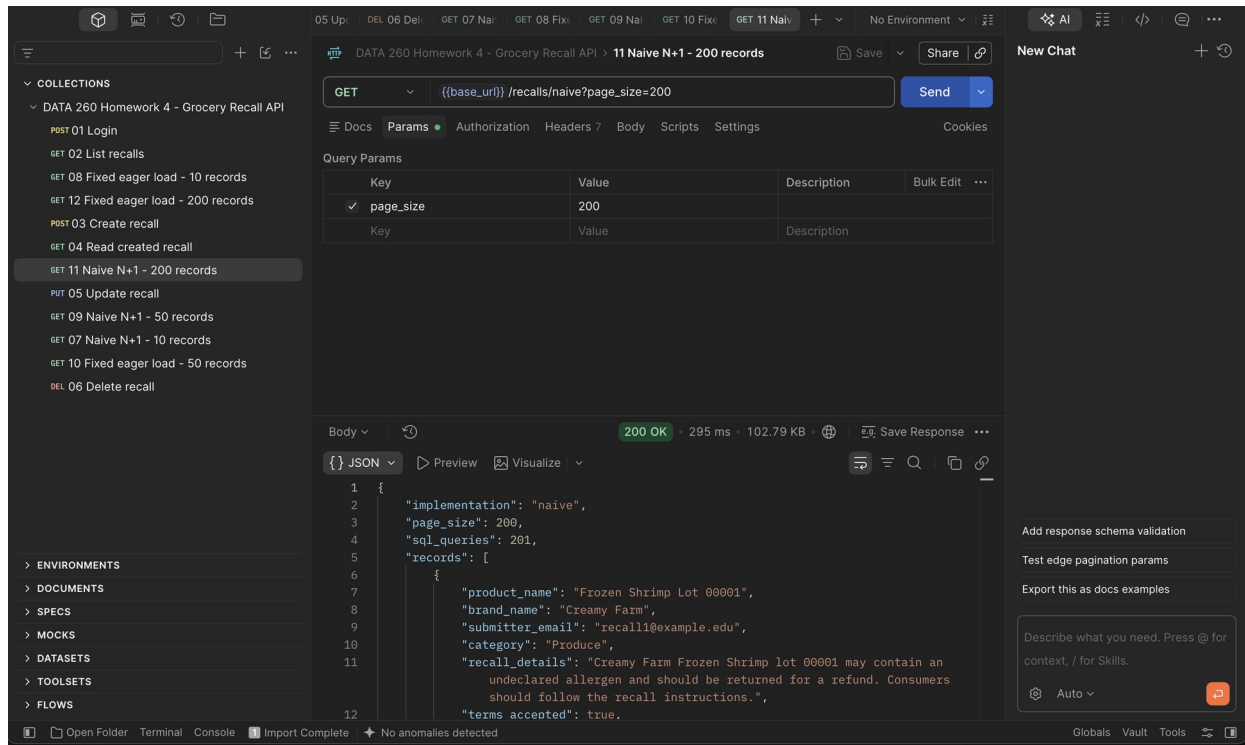


Figure 21. Naive endpoint at page size 200 with 201 SQL queries.

3.6 Fixed endpoint at page size 200

The fixed endpoint still performs one statement even when 200 records are returned.

Measured request

GET /api/v1/recalls/fixed?page_size=200

Expected SQL statements: 1

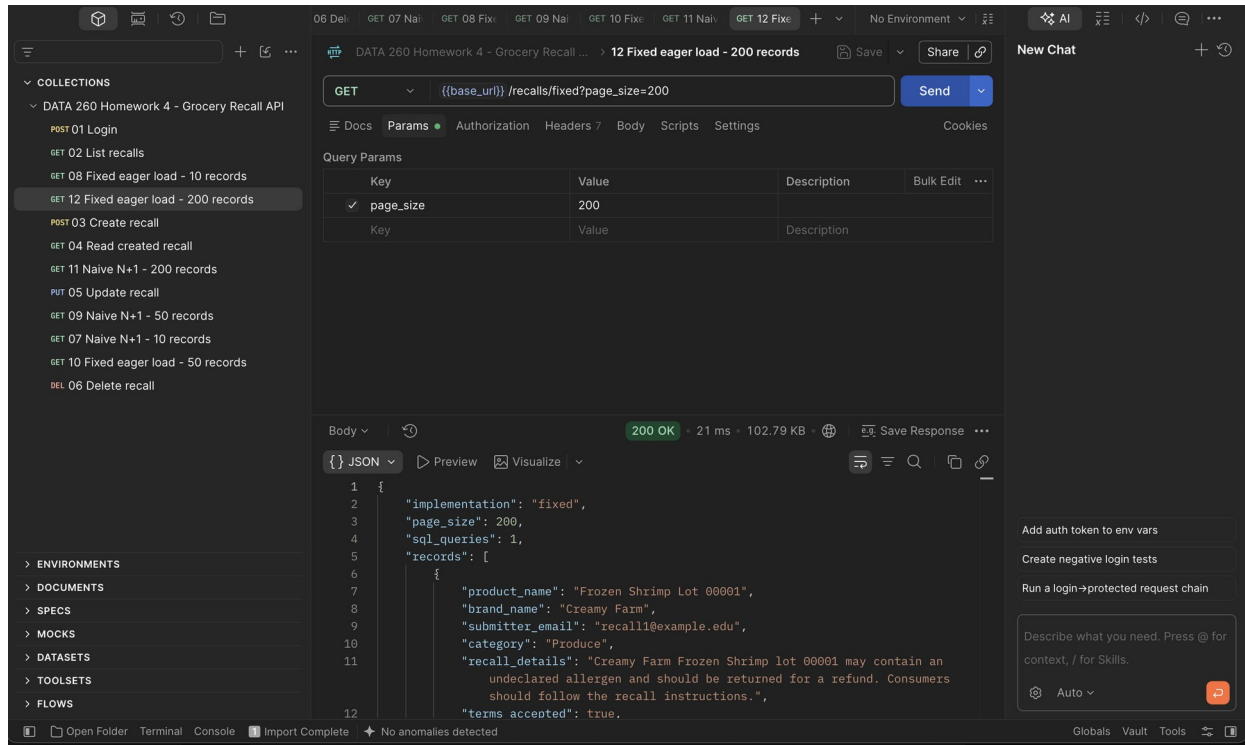


Figure 22. Fixed endpoint at page size 200 with one SQL query.

3.7 Latency and SQL statement results

Experiment loop

```
for page_size in PAGE_SIZES:
    for implementation in IMPLEMENTATIONS:
        endpoint = f"/api/v1/recalls/{implementation}"
        for run_number in range(1, RUNS_PER_CASE + 1):
            started = time.perf_counter()
            response = client.get(endpoint, params={"page_size": page_size})
            latency_ms = (time.perf_counter() - started) * 1000
            response.raise_for_status()
            body = response.json()
            rows.append(
                {
                    "timestamp_utc": datetime.now(timezone.utc).isoformat(),
                    "implementation": implementation,
                    "page_size": page_size,
                    "run": run_number,
                    "sql_queries": int(response.headers["X-SQL-Query-Count"]),
                    "latency_ms": round(latency_ms, 3),
                    "records_returned": len(body["records"]),
                }
            )
        )
```

Page size	Version	SQL/req	p50 ms	p95 ms	p99 ms	Speed-up
10	naive	11	6.023	7.935	8.608	-
10	fixed	1	3.303	8.523	25.920	1.82x
50	naive	51	18.035	20.308	22.296	-
50	fixed	1	5.656	7.242	19.120	3.19x
200	naive	201	59.150	72.617	74.108	-
200	fixed	1	11.273	14.457	20.258	5.25x

The fixed median speed-up grows from 1.82x at 10 rows to 5.25x at 200 rows because the naive version adds one round trip and one query execution for every returned record. The fixed query count stays constant, so its database overhead grows much more slowly. The p95 and p99 values still vary because local scheduling, connection reuse, serialization, and warm-up noise affect individual requests; the median and query counts show the intended scaling behavior most clearly.

```

# database_evidence.txt  # project_structure.txt  METRICS.md X
Users > vedantvartak > Documents > Codex > 2026-08-25 > c > outputs > data260-3539 > reports > hw04 > METRICS.md > # Homework 4 Metrics

1 # Homework 4 Metrics
2
3 Model for Part 4: 'qwen3:8b'; SEED: '3539'; VERIFY_SEED: '263539'.
4
5 ## N+1 query comparison
6
7 | Page size | Version | SQL statements/request | p50 (ms) | p95 (ms) | p99 (ms) | Speed-up |
8 | ---: | ---: | ---: | ---: | ---: | ---: | ---: |
9 | 10 | naive | 11 | 6.023 | 7.935 | 8.688 | - |
10 | 10 | fixed | 1 | 3.303 | 8.523 | 25.920 | 1.82x |
11 | 50 | naive | 51 | 18.035 | 20.308 | 22.296 | - |
12 | 50 | fixed | 1 | 5.656 | 7.242 | 19.120 | 3.19x |
13 | 200 | naive | 281 | 59.150 | 72.617 | 74.188 | - |
14 | 200 | fixed | 1 | 11.273 | 14.457 | 20.258 | 5.25x |
15
16 The naive endpoint issues one recall query plus one related-event query per returned record. The fixed endpoint uses eager loading and completes the same response with one SQL statement. The difference grows as the naive query count grows linearly while the fixed query count stays constant.
17
18 ## Index experiment
19
20 | Plan | Access type | Key | Estimated rows |
21 | ---: | ---: | ---: | ---: |
22 | Before index | ALL | - | 4967 |
23 | After index | ref | idx_recall_notices_category | 1188 |
24
25 The category filter changed from a full table scan ('ALL') to indexed reference access ('ref').
26
27 ## Grounded RAG evaluation
28
29 | Configuration | Correct answers | Grounded answers | Format compliant |
30 | ---: | ---: | ---: | ---: |
31 | no_rag | 1/6 | 0/6 | 4/6 |
32 | basic_rag | 4/6 | 0/6 | 4/6 |
33 | context_rag | 6/6 | 6/6 | 6/6 |
34
35 The context-engineered configuration cited retrieved sources and used the required refusal for both unsupported questions.
36
37 ## top-k comparison for the two-chunk question
38
39 | k | Chunks kept | Correct retrieval | Correct answer |
40 | ---: | ---: | ---: | ---: |
41 | 1 | 1 | No | No |
42 | 3 | 3 | No | No |
43 | 5 | 5 | Yes | Yes |
44

```

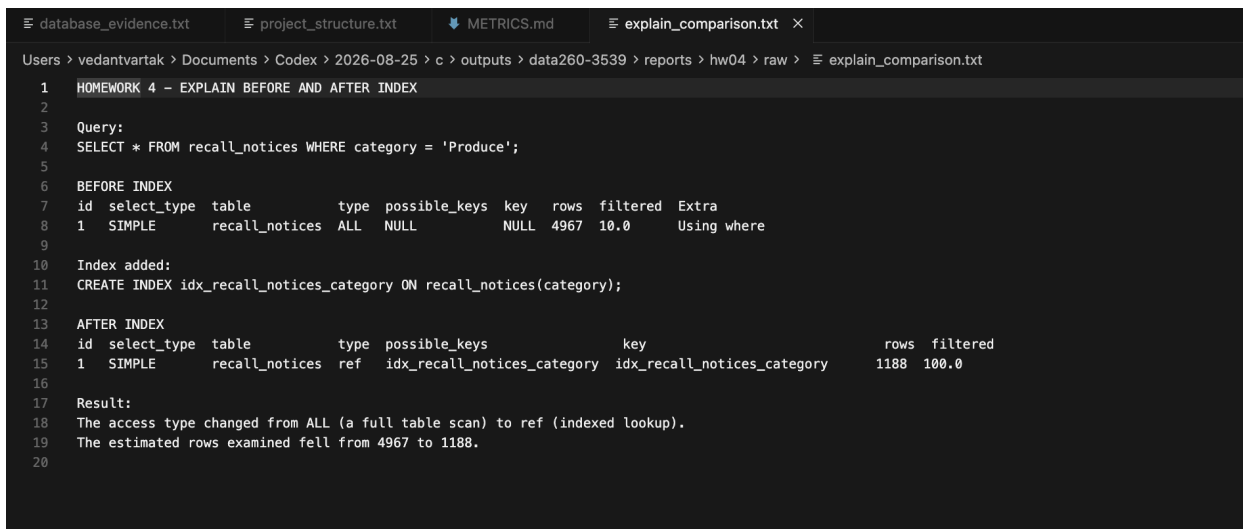
Figure 23. Saved N+1 metrics, index result, and RAG summary.

3.8 Index and EXPLAIN comparison

The experiment drops any earlier copy of the index, captures EXPLAIN, creates an index on category, and captures EXPLAIN again.

```
run_hw4_explain.py
def main() -> None:
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    with engine.begin() as connection:
        existing = connection.execute(
            text(
                "SELECT COUNT(*) FROM information_schema.statistics "
                "WHERE table_schema = DATABASE() "
                "AND table_name = 'recall_notices' "
                "AND index_name = :index_name"
            ),
            {"index_name": INDEX_NAME},
        ).scalar_one()
        if existing:
            connection.execute(text(f"DROP INDEX {INDEX_NAME} ON recall_notices"))

        before = rows_as_dicts(connection, EXPLAIN_SQL)
        connection.execute(
            text(f"CREATE INDEX {INDEX_NAME} ON recall_notices (category)")
        )
        after = rows_as_dicts(connection, EXPLAIN_SQL)
```



```
database_evidence.txt project_structure.txt METRICS.md explain_comparison.txt x
Users > vedantvartak > Documents > Codex > 2026-08-25 > c > outputs > data260-3539 > reports > hw04 > raw > explain_comparison.txt

1  HOMEWORK 4 - EXPLAIN BEFORE AND AFTER INDEX
2
3  Query:
4  SELECT * FROM recall_notices WHERE category = 'Produce';
5
6  BEFORE INDEX
7  id select_type table type possible_keys key rows filtered Extra
8  1 SIMPLE recall_notices ALL NULL NULL 4967 10.0 Using where
9
10 Index added:
11 CREATE INDEX idx_recall_notices_category ON recall_notices(category);
12
13 AFTER INDEX
14 id select_type table type possible_keys key rows filtered
15 1 SIMPLE recall_notices ref idx_recall_notices_category idx_recall_notices_category 1188 100.0
16
17 Result:
18 The access type changed from ALL (a full table scan) to ref (indexed lookup).
19 The estimated rows examined fell from 4967 to 1188.
20
```

Figure 24. EXPLAIN before and after adding `idx_recall_notices_category`.

Before the index, MySQL used access type ALL, no key, and estimated 4,967 rows, which indicates a full table scan. After the index, the access type changed to ref, MySQL selected `idx_recall_notices_category`, and the estimate fell to 1,188 rows. The query can now locate rows through the category index instead of scanning the full table.

Part 4 Grounded RAG question answering

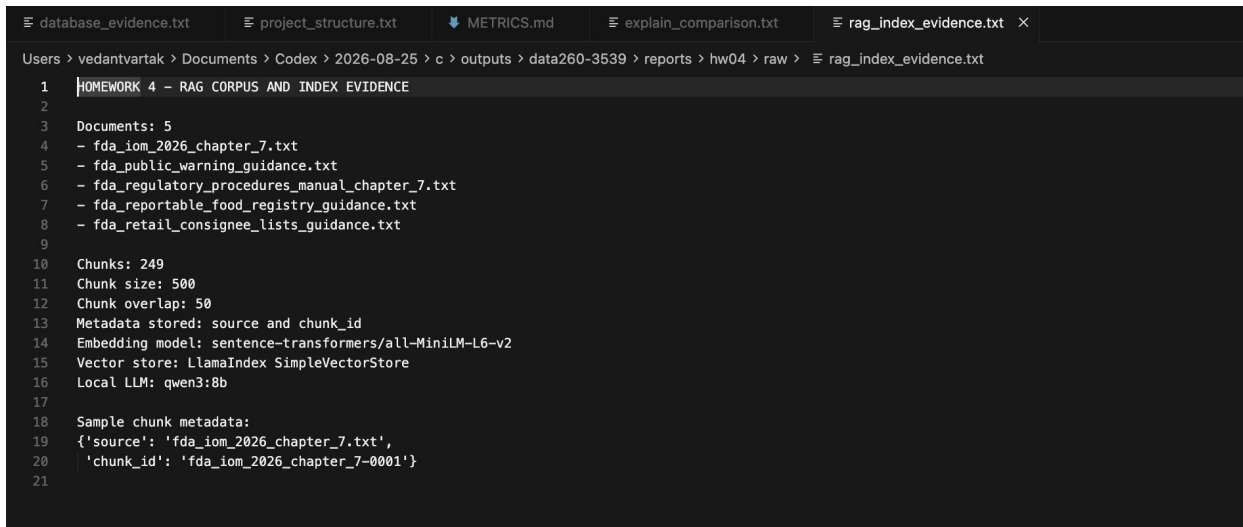
The RAG system uses five FDA recall documents, 500-token chunks with 50-token overlap, source and chunk identifiers, all-MiniLM-L6-v2 embeddings, a LlamaIndex SimpleVectorStore, and qwen3:8b. The same six questions are evaluated with no RAG, basic top-3 RAG, and context-engineered RAG.

4.1 Corpus, chunks, metadata, and vector index

Every document stores its source name. Each generated node receives a stable `chunk_id` before it is embedded and loaded into the vector store.

```
rag.py
def load_documents() -> list[Document]:
    documents = []
    for path in sorted(CORPUS_DIR.glob("*.txt")):
        document = Document(
            text=path.read_text(encoding="utf-8"),
            metadata={"source": path.name},
        )
        document.excluded_embed_metadata_keys = ["source"]
        documents.append(document)
    if len(documents) < 5:
        raise ValueError("HW4 requires a corpus of at least five documents")
    return documents

def build_index() -> tuple[VectorStoreIndex, list[Any], HuggingFaceEmbedding]:
    embed_model = HuggingFaceEmbedding(
        model_name=EMBEDDING_MODEL,
        cache_folder=str(MODEL_CACHE),
        normalize=True,
        show_progress_bar=False,
    )
    splitter = TokenTextSplitter(chunk_size=500, chunk_overlap=50)
    nodes = splitter.get_nodes_from_documents(load_documents(), show_progress=False)
    for position, node in enumerate(nodes, start=1):
        source = str(node.metadata.get("source", "unknown"))
        node.metadata["chunk_id"] = f"{Path(source).stem}-{position:04d}"
        node.excluded_embed_metadata_keys = ["source", "chunk_id"]
    storage = StorageContext.from_defaults(vector_store=SimpleVectorStore())
    index = VectorStoreIndex(
        nodes=nodes,
        embed_model=embed_model,
        storage_context=storage,
        show_progress=False,
    )
    return index, nodes, embed_model
```



```
database_evidence.txt  project_structure.txt  METRICS.md  explain_comparison.txt  rag_index_evidence.txt X
Users > vedantvartak > Documents > Codex > 2026-08-25 > c > outputs > data260-3539 > reports > hw04 > raw > rag_index_evidence.txt
1  HOMEWORK 4 - RAG CORPUS AND INDEX EVIDENCE
2
3  Documents: 5
4  - fda_iom_2026_chapter_7.txt
5  - fda_public_warning_guidance.txt
6  - fda_regulatory_procedures_manual_chapter_7.txt
7  - fda_reportable_food_registry_guidance.txt
8  - fda_retail_consignee_lists_guidance.txt
9
10 Chunks: 249
11 Chunk size: 500
12 Chunk overlap: 50
13 Metadata stored: source and chunk_id
14 Embedding model: sentence-transformers/all-MiniLM-L6-v2
15 Vector store: LlamaIndex SimpleVectorStore
16 Local LLM: qwen3:8b
17
18 Sample chunk metadata:
19 {'source': 'fda_iom_2026_chapter_7.txt',
20  'chunk_id': 'fda_iom_2026_chapter_7-0001'}
```

Figure 25. Five-document corpus, 249 chunks, metadata, embedding model, and vector store.

4.2 Top-k retrieval output

Retrieval records rank, score, source, chunk ID, text, and latency before the model is called. The printed evidence covers all six question types.

```
rag.py
def retrieve(index: VectorStoreIndex, question: str, top_k: int) -> list[dict[str, Any]]:
    started = perf_counter()
    matches = index.as_retriever(similarity_top_k=top_k).retrieve(question)
    latency_ms = (perf_counter() - started) * 1000
    rows = []
    for rank, match in enumerate(matches, start=1):
        rows.append(
            {
                "rank": rank,
                "score": round(float(match.score or 0.0), 6),
                "source": str(match.node.metadata.get("source", "unknown")),
                "chunk_id": str(match.node.metadata.get("chunk_id", match.node.node_id)),
                "text": match.node.get_content(metadata_mode=MetadataMode.NONE).strip(),
                "retrieval_latency_ms": round(latency_ms, 3),
            }
        )
    return rows

def print_retrieval(question: dict[str, Any], chunks: list[dict[str, Any]], top_k: int) -> str:
    lines = [f"QUESTION {question['id']} | top_k={top_k} | {question['question']}"]
    for chunk in chunks:
        preview = " ".join(chunk["text"].split())[:260].rstrip()
        lines.append(
            f"rank={chunk['rank']} score={chunk['score']:.4f} "
            f"source={chunk['source']} chunk_id={chunk['chunk_id']}\n{preview}"
        )
    return "\n".join(lines)
```

```
database_evidence.txt project_structure.txt METRICS.md explain_comparison.txt rag_index_evidence.txt retrieval_printouts.txt X
Users > vedantvartak > Documents > Codex > 2026-08-25 > c > outputs > data260-3539 > reports > hw04 > raw > E retrieval_printouts.txt
1 QUESTION q1 | top_k=3 | How quickly must a responsible party report a reportable food to the FDA after making that determination?
2 rank=1 score=0.7192 source=fda_reportable_food_registry_guidance.txt chunk_id=fda_reportable_food_registry_guidance-8232
3 and 417(e)(9) of the FDSC Act, 2. Notifications If a responsible party (Party A) notifies the immediate previous source(s) and/or immediate subsequent recipient(s) of the article of food (Party B), as required by FDA, should Party B submit a report to FDA and
4 rank=2 score=0.7125 source=fda_reportable_food_registry_guidance.txt chunk_id=fda_reportable_food_registry_guidance-8235
5 This ICSR number will allow FDA to link reportable food reports that are related to the initial report and the reportable food. J. Additional Contacts Should responsible parties call the FDA district office and notify state and local public health or regulator
6 rank=3 score=0.6887 source=fda_reportable_food_registry_guidance.txt chunk_id=fda_reportable_food_registry_guidance-8233
7 I required to submit a report to the Reportable Food electronic portal? If you determine that a food is a reportable food, you must submit a reportable food report as soon as practicable, but in no case later than 24 hours after determining that the article of
8
9 QUESTION q2 | top_k=3 | What product-identifying details and consumer instructions should an FDA public recall warning contain?
10 rank=1 score=0.8371 source=fda_public_warning_guidance.txt chunk_id=fda_public_warning_guidance-8043
11 6 Firms are reminded that, under 21 CFR § 7.42, they need not delay the initiation of a recall pending the FDA's review of their recall strategy. — Page 9 — Contains Nonbinding Recommendations 9.3. What information should be contained in a public warning?
12 rank=2 score=0.7746 source=fda_regulatory_procedures_manual_chapter_7.txt chunk_id=fda_regulatory_procedures_manual_chapter_7-8089
13 for Industry and FDA Staff (POP – 7180). The purpose of this guidance is to assist and provide recommendations to industry and FDA staff regarding the use, content, and circumstances for issuance of public warnings and public notifications for firm-initiated o
14 rank=3 score=0.7729 source=fda_regulatory_procedures_manual_chapter_7.txt chunk_id=fda_regulatory_procedures_manual_chapter_7-8093
15 public warning when requested by the DRC – In instances where the firm's public warning appears inadequate, discuss with the CHU whether to post the firm's public warning to FDA's website – Post firm's, State's or FDA's recall public warning and images to the
16
17 QUESTION q3 | top_k=3 | How do public warnings and retail-consignee disclosures each help consumers respond to a food recall?
18 rank=1 score=0.8223 source=fda_retail_consignee_lists_guidance.txt chunk_id=fda_retail_consignee_lists_guidance-8245
19 Is relevant to the recall, whether a recall is voluntarily initiated or ordered by FDA. Under 21 CFR 20.91, information otherwise exempt from public disclosure, such as CCI, is nevertheless available for public disclosure to the extent necessary to effectuate
20 rank=2 score=0.8041 source=fda_retail_consignee_lists_guidance.txt chunk_id=fda_retail_consignee_lists_guidance-8246
21 after exposure to a contaminated food, while this guidance is specific to retail consignees, our authority under 21 CFR 20.91 is not limited to retail consignees, and can include the disclosure of the names of restaurants and similar entities where appropriate
22 rank=3 score=0.7896 source=fda_retail_consignee_lists_guidance.txt chunk_id=fda_retail_consignee_lists_guidance-8248
23 – especially when a recalled food is associated with a foodborne illness outbreak – when doing so will be of most use to consumers in identifying a recalled food and is consistent with 21 CFR 20.91. For example, FDA might post retail consignee information for
24
25 QUESTION q4 | top_k=3 | When should the FDA communicate publicly about a recall?
26 rank=1 score=0.7333 source=fda_public_warning_guidance.txt chunk_id=fda_public_warning_guidance-8047
27 When a firm posts its public warning on its own webpage, it should ensure that the warning is prominently displayed and accessible from both its home page and a web search. The posting should remain publicly accessible until the product is no longer expected t
28 rank=2 score=0.7247 source=fda_regulatory_procedures_manual_chapter_7.txt chunk_id=fda_regulatory_procedures_manual_chapter_7-8088
29 The division office will offer the same guidance to the recalling firm as outlined above in Firm Recall Communication and Notification and will assist the firm in arranging the text of recall communications to consignees so that the product will be promptly re
29 rank=3 score=0.7211 source=fda_public_warning_guidance.txt chunk_id=fda_public_warning_guidance-8046
31 that language and distributed via the appropriate distribution service. Posting a Public Warning or Press Release on a firm or FDA Webpage In many cases, a firm will announce a removal of an FDA regulated product from the market or correction through a press
32
33 QUESTION q5 | top_k=3 | What exact font size does the FDA require on every grocery recall poster?
34 rank=1 score=0.5229 source=fda_retail_consignee_lists_guidance.txt chunk_id=fda_retail_consignee_lists_guidance-8243
35 Department of Agriculture (USDA) policy is to make publicly available the names and locations of retail consignees of recalled meat or poultry products that USDA compiles in connection with a recall where there is a reasonable probability that the use of the p
36 rank=2 score=0.5188 source=fda_retail_consignee_lists_guidance.txt chunk_id=fda_retail_consignee_lists_guidance-8245
37 Is relevant to the recall, whether a recall is voluntarily initiated or ordered by FDA. Under 21 CFR 20.91, information otherwise exempt from public disclosure, such as CCI, is nevertheless available for public disclosure to the extent necessary to effectuate
38 rank=3 score=0.5073 source=fda_regulatory_procedures_manual_chapter_7.txt chunk_id=fda_regulatory_procedures_manual_chapter_7-8086
39 self-addressed post card, envelope, or other arrangement to enable the consignee to report the amount of the product available and its disposition. Recall communications should direct that the consignee submit a report regardless of whether or not any of the
40
41 QUESTION q6 | top_k=3 | Which team won the 2026 World Series?
42 rank=1 score=0.1376 source=fda_ism_2026_chapter_7.txt chunk_id=fda_ism_2026_chapter_7-8033
43 Postage will be paid by: JOHN DOE LABORATORIES Somewhere, U.S.A. 12345-6789 Henry Doe — Page 23 — EXHIBIT 7-1 INVESTIGATIONS OPERATIONS MANUAL 2026 25 John Doe Laboratories first class mail A. B. C. Pharmacy Anywhere U. S. A. (red print) URGENT DRUG RECALL
44 rank=2 score=0.1833 source=fda_regulatory_procedures_manual_chapter_7.txt chunk_id=fda_regulatory_procedures_manual_chapter_7-8074
45 to confirm a violation or potential for health hazard, as necessary, 5. Refer the recall to an FDA Recall Coordinator for the appropriate Program Division. OSPOP/DE/ROB will consider available information when determining which Program Division should handle t
46 rank=3 score=0.1812 source=fda_regulatory_procedures_manual_chapter_7.txt chunk_id=fda_regulatory_procedures_manual_chapter_7-8208
47 Grocery corporate headquarters Manufacturer Repacker Hospital/Medical facility Pharmacy-retail Medical Laboratory Hospital pharmacies Retailer Other: — Page 131 — Regulatory Procedures Manual July 2021 Chapter 7 Recall Procedures MAN-000018 Page 131 of 153
48
```

Figure 26. Top-3 chunks, sources, scores, and IDs printed for all six questions.

4.3 No RAG, basic RAG, and context-engineered RAG

The context-engineered prompt labels sources, prohibits outside knowledge, requires citations, and gives the exact refusal sentence.

```
rag.py
def call_model(client: OllamaClient, configuration: str, question: str, chunks: list[dict[str, Any]]) -> str:
    if configuration == "no_rag":
        prompt = f"Answer the following question concisely.\n\nQuestion: {question}"
    elif configuration == "basic_rag":
        prompt = (
            "Use the retrieved text below to answer the question.\n\n"
            f"{format_context(chunks, labels=False)}\n\nQuestion: {question}"
        )
    else:
        context = format_context(chunks, labels=True)
        prompt = (
            "Answer only from the supplied context. Do not use outside knowledge. "
            "Cite supporting evidence with [Source N]. If the context does not explicitly "
            f"support an answer, reply exactly: {REFUSAL}\n\n"
            f"Context:\n{context or '(no relevant context remained)}'\n\n"
            f"Question: {question}"
        )
    result = client.complete(
        [{"role": "user", "content": prompt}],
        temperature=0.0,
    )
    return result.content.strip()
```

The screenshot shows a code editor with a file named `rag_configuration_comparison.txt`. The file contains a table comparing three RAG configurations: No RAG, Basic RAG, and Context RAG. The table has four columns: Configuration, Correct answers, Grounded answers, and Format compliant. Below the table, the results for two questions (Q5 and Q6) are shown for each configuration. For Q5, the Context RAG configuration provides an exact refusal sentence, while the other two configurations do not. For Q6, the Context RAG configuration also provides an exact refusal sentence, while the other two configurations do not.

Configuration	Correct answers	Grounded answers	Format compliant
No RAG	1/6	0/6	4/6
Basic RAG	4/6	0/6	4/6
Context RAG	6/6	6/6	6/6

Q5 - Information not in the documents
Question: What exact font size does the FDA require on every grocery recall poster?

NO RAG:
The FDA requires a font size of at least 18 points on every grocery recall poster.
Result: incorrect hallucination

BASIC RAG:
The provided text does not mention any specific font size requirements for FDA recall posters. Therefore, the exact font size cannot be determined.
Result: reasonable, but it did not use the required exact refusal

CONTEXT RAG:
I cannot answer this question from the provided documents
Result: PASS - exact refusal

Q6 - Unrelated question
Question: Which team won the 2026 World Series?

NO RAG:
As of now, the 2026 World Series has not occurred, and no team has been declared.
Result: answered from outside knowledge instead of refusing

BASIC RAG:
The provided text does not contain any information about the 2026 World Series.
Result: reasonable, but it did not use the required exact refusal

CONTEXT RAG:
I cannot answer this question from the provided documents
Result: PASS - exact refusal

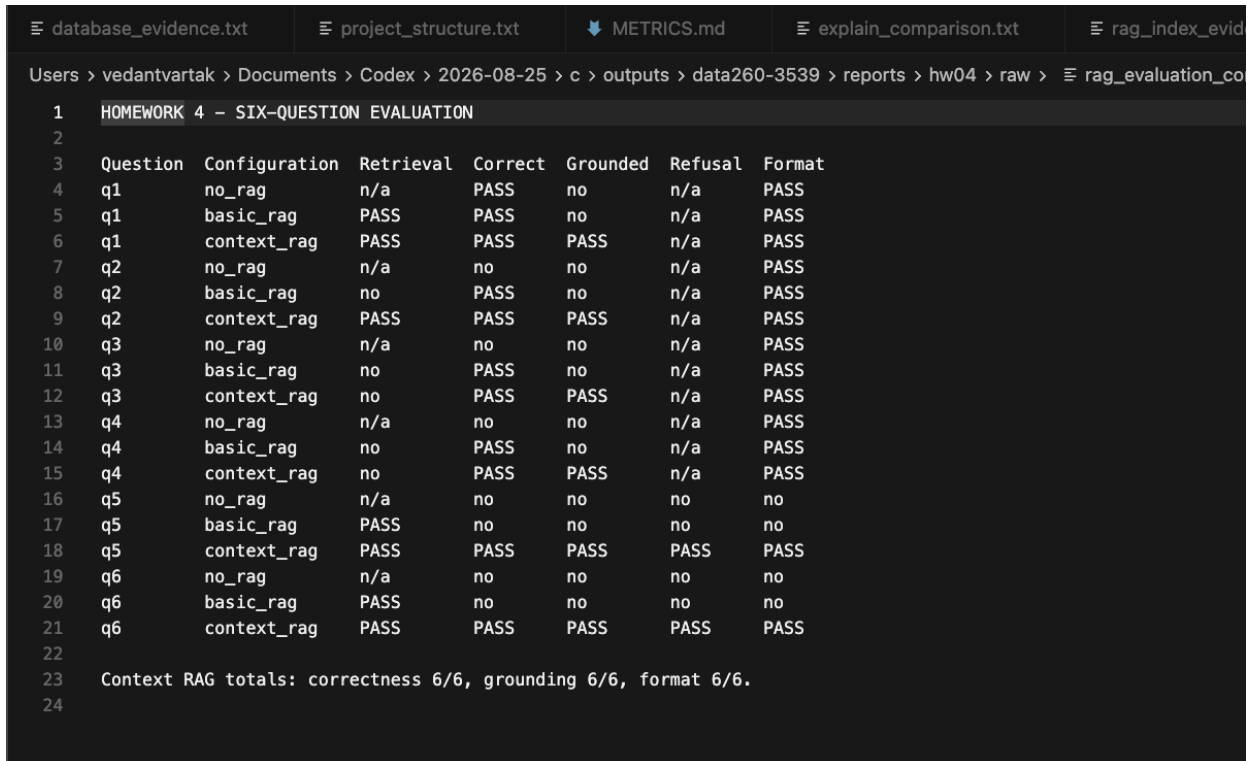
Figure 27. Three-configuration comparison and exact refusals for Q5 and Q6.

4.4 Six-question evaluation

Evaluation records retrieval, correctness, grounding, refusal behavior, and format compliance for every question and configuration.

```
rag.py
def answer_is_correct(question: dict[str, Any], answer: str) -> bool:
    if question["should_refuse"]:
        return REFUSAL.lower() in answer.lower()
    lowered = answer.lower()
    return all(keyword.lower() in lowered for keyword in question["expected_keywords"])

def evaluate(question: dict[str, Any], configuration: str, answer: str, chunks: list[dict[str, Any]]) -> dict[str, Any]:
    correct_retrieval = retrieval_is_correct(question, chunks) if configuration != "no_rag" else None
    correct_answer = answer_is_correct(question, answer)
    refused = REFUSAL.lower() in answer.lower()
    grounded = (
        configuration == "context_rag"
        and correct_answer
        and (refused if question["should_refuse"] else "[source " in answer.lower())
    )
    format_compliant = (
        refused if question["should_refuse"]
        else configuration != "context_rag" or "[source " in answer.lower()
    )
    return {
        "question_id": question["id"],
        "question_type": question["type"],
        "configuration": configuration,
        "correct_retrieval": correct_retrieval,
        "correct_answer": correct_answer,
        "grounded": grounded,
        "refused_when_needed": refused if question["should_refuse"] else None,
        "format_compliant": format_compliant,
        "answer": answer,
    }
```



The screenshot shows a code editor interface with a file explorer at the top containing files like 'database_evidence.txt', 'project_structure.txt', 'METRICS.md', 'explain_comparison.txt', and 'rag_index_evid'. The active file is 'rag_evaluation_co'. Below the file explorer, a terminal window displays the following evaluation table:

	Question	Configuration	Retrieval	Correct	Grounded	Refusal	Format
1	HOMEWORK 4 - SIX-QUESTION EVALUATION						
2							
3	q1	no_rag	n/a	PASS	no	n/a	PASS
4	q1	basic_rag	PASS	PASS	no	n/a	PASS
5	q1	context_rag	PASS	PASS	PASS	n/a	PASS
6	q2	no_rag	n/a	no	no	n/a	PASS
7	q2	basic_rag	no	PASS	no	n/a	PASS
8	q2	context_rag	PASS	PASS	PASS	n/a	PASS
9	q3	no_rag	n/a	no	no	n/a	PASS
10	q3	basic_rag	no	PASS	no	n/a	PASS
11	q3	context_rag	no	PASS	PASS	n/a	PASS
12	q4	no_rag	n/a	no	no	n/a	PASS
13	q4	basic_rag	no	PASS	no	n/a	PASS
14	q4	context_rag	no	PASS	PASS	n/a	PASS
15	q5	no_rag	n/a	no	no	no	no
16	q5	basic_rag	PASS	no	no	no	no
17	q5	context_rag	PASS	PASS	PASS	PASS	PASS
18	q6	no_rag	n/a	no	no	no	no
19	q6	basic_rag	PASS	no	no	no	no
20	q6	context_rag	PASS	PASS	PASS	PASS	PASS
21							
22							
23	Context RAG totals: correctness 6/6, grounding 6/6, format 6/6.						
24							

Figure 28. Evaluation table with context-RAG totals of 6/6 for correctness, grounding, and format.

4.5 Evaluation summary

Configuration	Correct answers	Grounded answers	Format compliant
No RAG	1/6	0/6	4/6
Basic RAG	4/6	0/6	4/6
Context RAG	6/6	6/6	6/6

Context RAG was the only configuration that was fully correct, cited the supplied evidence, and refused both unsupported questions with the required wording.

4.6 Context-size sweep

The two-chunk question was repeated at $k=1$, $k=3$, and $k=5$. The larger setting supplied the second required public-warning chunk, while also adding a less relevant retail-consignee chunk.

```
rag.py
    }
    results.append(row)
    evaluations.append(evaluate(question, configuration, answer, chunks))
    print(f"\n{configuration.upper()} ANSWER\n{answer}")

sweep_question = questions[1]
sweep_rows = []
for top_k in (1, 3, 5):
    chunks = retrieve(index, sweep_question["question"], top_k)
    engineered = engineer_context(chunks)
    answer = call_model(client, "context_rag", sweep_question["question"], engineered)
    sweep_rows.append(
        {
            "question_id": sweep_question["id"],
            "top_k": top_k,
            "retrieved_chunks": chunks,
            "chunks_after_engineering": len(engineered),
```

```
database_evidence.txt project_structure.txt METRICS.md explain_comparison.txt rag_index_evidence.txt
Users > vedantvartak > Documents > Codex > 2026-08-25 > c > outputs > data260-3539 > reports > hw04 > raw > rag_k_sweep_evidence
1  HOMEWORK 4 - TOP_K SWEEP FOR THE TWO-CHUNK QUESTION
2
3  Question q2:
4  What product-identifying details and consumer instructions should an FDA public
5  recall warning contain?
6
7  k  kept  correct retrieval  correct answer
8  1  1      No              No
9  3  3      No              No
10 5  5      Yes             Yes
11
12 k=1
13 rank 1 score 0.8371 fda_public_warning_guidance-0043
14 Only one part of the answer was available.
15
16 k=3
17 rank 1 score 0.8371 fda_public_warning_guidance-0043
18 rank 2 score 0.7746 fda_regulatory_procedures_manual_chapter_7-0089
19 rank 3 score 0.7729 fda_regulatory_procedures_manual_chapter_7-0092
20 The second required public-warning chunk was still missing.
21
22 k=5
23 rank 1 score 0.8371 fda_public_warning_guidance-0043
24 rank 2 score 0.7746 fda_regulatory_procedures_manual_chapter_7-0089
25 rank 3 score 0.7729 fda_regulatory_procedures_manual_chapter_7-0092
26 rank 4 score 0.7600 fda_public_warning_guidance-0047
27 rank 5 score 0.7396 fda_retail_consignee_lists_guidance-0243
28
29 Conclusion:
30 k=5 was best because rank 4 supplied the second needed public-warning chunk.
31 It also introduced a less relevant retail-consignee chunk at rank 5, showing that
32 more context can add noise even when it improves retrieval completeness.
33 |
```

Figure 29. Top-k sweep showing $k=5$ as the first complete and correct setting.

4.7 RAG comparison analysis

I compared three versions of the same six-question test: the local model by itself, basic RAG, and RAG with context engineering. The questions were intentionally mixed. One could be answered from a single chunk, one needed details spread across more than one chunk, two involved similar or ambiguous recall guidance, one asked for information that was not in the documents, and one was unrelated to the corpus.

The no-RAG version answered only 1 of 6 questions correctly. Its clearest failure was inventing an 18-point FDA font requirement even though the documents never gave one. Basic RAG improved to 4 of 6 because retrieved FDA text supplied the missing factual details. It still did not follow the exact refusal rule for the two unsupported questions, however, and its answers did not include source citations. This made the responses more informative than the no-RAG version but not consistently grounded or format compliant.

The context-engineered version scored 6 of 6 for correctness, grounding, and format compliance. I filtered low-relevance chunks, removed near-duplicate passages, labeled each retained chunk with its source file and chunk ID, and instructed the model to use only that context. It cited sources for answerable questions and returned the required refusal sentence for both the missing-information and unrelated questions. This directly prevented the unsupported font-size claim seen without RAG.

The top-k comparison showed why retrieval depth matters. The two-chunk question failed with k=1 and k=3 because the retrieved context did not contain enough of the expected evidence. With k=5, retrieval and the final answer were both correct. A larger k is not automatically better because it can add distracting passages, so I used filtering and deduplication before giving the context to the model.

The main remaining limitation is that automated keyword and source checks do not measure every aspect of answer quality. A response can include the expected terms while still being vague, and a relevant passage can rank below similar FDA language from another document. For this small local corpus, k=5 with context filtering produced the best balance. In a larger system I would add manually graded questions, citation verification, and a reranker before treating the evaluation as production evidence.

Word count: 361

Reproducibility and self-check

The repository contains the schema, deterministic seeder, React and FastAPI code, Postman collection, 180-request raw N+1 CSV and JSON files, EXPLAIN outputs, RAG results, retrieval printouts, evaluation data, METRICS.md, RUN_LOG.txt, and verification.json.

Smoke-test entry point

```
def main() -> None:
    checks: list[dict[str, object]] = []
    process = subprocess.Popen(
        [
            sys.executable,
            "-m",
            "uvicorn",
            "code.web_application.backend:app",
            "--host",
            "127.0.0.1",
            "--port",
            str(PORT_BASE),
        ],
        cwd=ROOT,
        env={**os.environ, "SESSION_COOKIE_SECURE": "false"},
        stdout=subprocess.DEVNULL,
        stderr=subprocess.DEVNULL,
    )
    try:
        with httpx.Client(base_url=f"http://127.0.0.1:{PORT_BASE}", timeout=20.0) as client:
            wait_for_server(client)
    ...
    with db_session_basede26() as db:
        recall_count = int(db.scalar(text("SELECT COUNT(*) FROM recall_notices"))) or 0
        event_count = int(db.scalar(text("SELECT COUNT(*) FROM recall_events"))) or 0
        add_check(checks, "seeded_primary_rows", recall_count == 5000, f"rows={recall_count}")
        add_check(checks, "seeded_related_rows", event_count == 200, f"rows={event_count}")

    request_csv = ROOT / "reports/hw04/raw/n_plus_one_requests.csv"
    raw_count = 0
    if request_csv.exists():
        with request_csv.open(encoding="utf-8") as handle:
            raw_count = sum(1 for _ in csv.DictReader(handle))
    add_check(checks, "raw_n_plus_one_requests", raw_count == 180, f"rows={raw_count}")

    evaluation_path = ROOT / "reports/hw04/raw/evaluation.json"
    evaluation = json.loads(evaluation_path.read_text()) if evaluation_path.exists() else []
    refusal_rows = [
        row for row in evaluation
        if row.get("configuration") == "context_rag" and row.get("question_id") in {"q5", "q6"}
    ]
    add_check(
        checks,
        "rag_q5_q6_refusals",
        len(refusal_rows) == 2 and all(row.get("refused_when_needed") for row in refusal_rows),
        f"passing refusals={sum(bool(row.get('refused_when_needed')) for row in refusal_rows)}/2",
    )

    frontend_index = ROOT / "code/web_application/frontend/dist/index.html"
    add_check(checks, "react_production_build", frontend_index.exists(), str(frontend_index.relative_to(ROOT)))
```

Check	Result	Details
fastapi_health	PASS	HTTP 200
login_required	PASS	HTTP 401 without a session
database_session_cookie	PASS	HTTP 200; HttpOnly/SameSite checked
naive_list_endpoint	PASS	HTTP 200; queries=11
fixed_list_endpoint	PASS	HTTP 200; queries=1
n_plus_one_removed	PASS	naive=11, fixed=1
seeded_primary_rows	PASS	rows=5000
seeded_related_rows	PASS	rows=200

Check	Result	Details
raw_n_plus_one_requests	PASS	rows=180
rag_q5_q6_refusals	PASS	passing refusals=2/2
react_production_build	PASS	code/web_application/frontend/dist/index.html

Result: 11 of 11 objective checks passed. verification.json records code-and-results commit 8753d1f50e8e9384df50d30a1277c095ce85cbde, so the smoke-test evidence identifies the submitted implementation.

AI-use disclosure

1. What I used assistance for and what I did myself

I used an AI assistant to help organize the assignment requirements, draft parts of the React, FastAPI, MySQL, and RAG implementation, and review the repository structure. I checked the assignment again, ran the application and experiments, tested the required operations, reviewed the saved outputs, captured the screenshots, and checked the final code and report.

2. One unsuitable output or independently verified item

For the question asking what exact font size the FDA requires on every grocery recall poster, the no-RAG configuration claimed that the FDA requires at least 18-point text. That answer was not supported by any of the provided documents.

3. How I detected or verified it

I compared the answer with the retrieved chunks and searched the five-document corpus for a supporting font-size requirement. The documents did not provide one, so I marked the no-RAG response as an incorrect hallucination in the evaluation results.

4. What I changed and why it works now

I added context-grounding instructions, source labels, and the required exact refusal for questions that the retrieved documents do not support. The context-RAG configuration then returned "I cannot answer this question from the provided documents" for both unsupported questions and achieved 6/6 for correctness, grounding, and format compliance.